

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه بناب
دانشکده علوم پایه
گروه ریاضی و علوم کامپیوتر

پایان نامه کارشناسی رشته علوم کامپیوتر

حل مسئله مستقیم معادله غیرخطی برگرز با استفاده از شبکه های
عصبی آگاه از فیزیک

استاد راهنما:
دکتر بابک آذرنوید

پژوهشگر:
محمد امیر بابازاد
شماره دانشجویی: ۱۴۰۲۱۳۵۱۰۴

شهریور ۱۴۰۵

کلیه حقوق مادی و معنوی مرتبط با نتایج مطالعات، ابتکارات و نوآوری‌های ناشی از تحقیق موضوع این پایان‌نامه متعلق به دانشگاه بناب است.

تقدیم به

پدر و مادر عزیزم،
که هرچه دارم از دعای خیر و ازخودگذشتگی آنهاست،
و به همه آنان که چراغ دانش را در این سرزمین روشن نگه داشته‌اند.

سپاسگزاری

سپاس بی‌کران خداوند یکتا را که توفیق انجام این پژوهش را به این‌جانب عطا فرمود. بر خود لازم می‌دانم از استاد ارجمند، جناب آقای دکتر بابک آذرنوید که راهنمایی این پایان‌نامه را بر عهده داشتند و در تمام مراحل انجام آن، از انتخاب موضوع تا تدوین نهایی، با حوصله و دقت راهنمایم بودند، صمیمانه قدردانی کنم. بی‌شک بدون رهنمودها و حمایت‌های ایشان، این پژوهش به سرانجام نمی‌رسید. همچنین از همه استادان گروه علوم کامپیوتر دانشگاه بناب که در دوران تحصیل از دانش و تجربه آنان بهره‌مند شدم، تشکر و قدردانی می‌کنم. در پایان، از پدر و مادر مهربانم که همواره مشوق و پشتیبان من در مسیر علم‌آموزی بوده‌اند و از همه دوستان و عزیزانی که به هر شکل مرا در انجام این پژوهش یاری رساندند، سپاسگزارم.

محمد امیر بابازاد

شهریور ۱۴۰۵

چکیده

حل عددی معادلات دیفرانسیل جزئی غیرخطی، یکی از چالش‌های بنیادین در دینامیک سیالات و محاسبات علمی است. روش‌های عددی کلاسیک با وجود برخورداری از دقت بالا، وابستگی شدیدی به تولید شبکه محاسباتی دارند و در مواجهه با هندسه‌های پیچیده یا ابعاد بالا با رشد سریع هزینه محاسباتی روبه‌رو می‌شوند. از سوی دیگر، مدل‌های یادگیری عمیق صرفاً داده‌محور نیازمند حجم عظیمی از داده‌های آموزشی تجربی بوده و در غیاب داده‌های متراکم قادر به تضمین سازگاری با اصول فیزیکی حاکم بر سیستم نیستند. چارچوب «شبکه‌های عصبی آگاه از فیزیک» (PINN) به‌عنوان رهیافتی نوین، قوانین دیفرانسیلی فیزیک را مستقیماً در ساختار آموزش شبکه عصبی تلفیق می‌نماید. هدف اصلی این پایان‌نامه، تحلیل نظری، پیاده‌سازی محاسباتی و ارزیابی دقیق این چارچوب در حل مسئله مستقیم معادله غیرخطی برگرز یک‌بعدی و مقایسه نتایج آن با مراجع معتبر علمی است.

در این پژوهش، پس از تشریح مبانی نظری معادلات دیفرانسیل جزئی، یادگیری عمیق و سازوکار مشتق‌گیری خودکار، ساختار مدل زمان‌پیوسته شبکه عصبی آگاه از فیزیک تبیین شده است. در بخش محاسباتی، مدل زمان‌پیوسته با معماری پرسپترون چندلایه در زبان برنامه‌نویسی پایتون پیاده‌سازی گردید.

ارزیابی‌های عددی و مقایسه جامع با نتایج مقاله مرجع معتبر رئیسی و همکاران (۲۰۱۹) نشان می‌دهد که شبکه عصبی آگاه از فیزیک قادر است تنها با استفاده از تعداد اندکی نقطه روی شرایط مرزی و اولیه و ارضای باقیمانده فیزیکی روی نقاط هم‌مکانی، میدان پیوسته پاسخ زمانی-مکانی معادله برگرز را با دقت بالا بازسازی نماید و پدیده تشکیل موج شوک را به خوبی ردیابی کند. یافته‌های این پژوهش بیانگر کارایی محاسباتی برجسته این رهیافت بدون نیاز به تولید شبکه محاسباتی و پتانسیل بالای آن در مدل‌سازی رفتارهای غیرخطی است.

کلیدواژه‌ها: شبکه‌های عصبی آگاه از فیزیک، معادله غیرخطی برگرز، مسئله مستقیم، مشتق‌گیری خودکار، پدیده تشکیل موج شوک

فهرست اختصارات

در این پایان نامه به منظور سهولت نگارش و اختصار متن، از کوتاه نوشت های استاندارد زیر استفاده شده است:

علامت اختصاری	شرح فارسی
ANN	شبکه عصبی مصنوعی
BC	شرط مرزی
CPU	واحد پردازش مرکزی
GPU	واحد پردازش گرافیکی
IC	شرط اولیه
L-BFGS	روش شبه نیوتنی با حافظه محدود
MLP	پرسپترون چندلایه
MSE	میانگین مربعات خطا
ODE	معادله دیفرانسیل معمولی
PDE	معادله دیفرانسیل جزئی
PINN	شبکه عصبی آگاه از فیزیک
ReLU	واحد خطی یکسوشده
RK	رانگه-کوتا
SSE	مجموع مربعات خطا

فهرست مطالب

پ	سپاسگزاری
ت	چکیده
ث	فهرست اختصارات
ح	فهرست تصاویر
خ	فهرست جداول
۱	۱ مقدمه و کلیات پژوهش
۳	۱-۱ طرح موضوع پژوهش
۳	۲-۱ بیان مسئله
۴	۳-۱ اهداف پژوهش
۵	۴-۱ پرسش‌های پژوهش
۵	۵-۱ روش‌شناسی پژوهش
۶	۶-۱ ساختار پایان‌نامه
۷	۲ مبانی نظری و پیشینه پژوهش
۷	۱-۲ معادلات دیفرانسیل جزئی
۹	۲-۲ روش‌های عددی کلاسیک
۱۰	۳-۲ شبکه‌های عصبی مصنوعی
۱۲	۴-۲ آموزش شبکه‌های عصبی عمیق
۱۳	۵-۲ پیشینه پژوهش
۱۴	۶-۲ جمع‌بندی فصل
۱۵	۳ شبکه‌های عصبی آگاه از فیزیک
۱۵	۱-۳ صورت‌بندی کلی مسئله و ایده محوری
۱۶	۲-۳ مبانی و سازوکار مشتق‌گیری خودکار
۱۸	۱-۲-۳ مبانی ریاضی و حساب اعداد دوگانه
۱۸	۲-۲-۳ حالت‌های مشتق‌گیری خودکار: مستقیم و معکوس
۱۹	۳-۳ به‌کارگیری مشتق‌گیری خودکار در حل معادله برگرز
۱۹	۱-۳-۳ گام اول: نمایش پارامتری میدان حالت با شبکه عصبی
	۲-۳-۳ گام دوم: استخراج مشتقات جزئی زمانی و مکانی با قاعده
۲۰	زنجره‌ای
۲۱	۳-۳-۳ گام سوم: تشکیل عملگر باقیمانده فیزیکی برگرز
	۴-۳-۳ گام چهارم: ساختار دو سطحی مشتق‌گیری خودکار در فرایند
۲۱	آموزش

۲۲	۳-۵ گام پنجم: سازوکار پیاده‌سازی تابعی در کتابخانه محاسباتی
۲۲	جکس
۲۲	۳-۴ مدل‌های زمان‌پیوسته برای مسئله مستقیم برگرز
۲۳	۳-۵ مدل‌های زمان‌گسسته
۲۴	۳-۶ ملاحظات عملی در پیاده‌سازی و آموزش
۲۵	۳-۷ جمع‌بندی فصل
۲۶	۴ پیاده‌سازی و ارزیابی نتایج
۲۶	۴-۱ بستر محاسباتی و ابزارهای پیاده‌سازی
۲۷	۴-۲ ساختار الگوریتمی و روند آموزش مدل با کتابخانه جکس
۲۷	۴-۲-۱ مقداردهی اولیه پارامترها با روش گزاویه
۲۷	۴-۲-۲ خط لوله تبدیلات تابعی در JAX
۲۸	۴-۲-۳ استراتژی بهینه‌سازی ترکیبی دو مرحله‌ای
۲۸	۴-۳ حل مستقیم معادله برگرز
۳۰	۴-۴ ارزیابی مقایسه‌ای با مقاله مرجع
۳۲	۴-۵ بحث و تحلیل نتایج
۳۳	۴-۶ جمع‌بندی فصل
۳۴	۵ نتیجه‌گیری و پیشنهادها
۳۴	۵-۱ خلاصه پژوهش
۳۵	۵-۲ یافته‌ها و دستاوردهای کلیدی
۳۵	۵-۳ پیشنهادها برای پژوهش‌های آینده
۳۷	فهرست منابع
۳۹	واژه‌نامه
۴۱	پیوست الف: کد پیاده‌سازی محاسباتی
۴۸	فرم ارزیابی پایان‌نامه
۴۹	Abstract

فهرست تصاویر

۱-۲	ساختار شماتیک یک شبکه عصبی پرسپترون چندلایه با دو متغیر ورودی، دو لایه پنهان و یک خروجی اسکالر	۱۱
۱-۳	نمودار بلوکی چارچوب شبکه عصبی آگاه از فیزیک: تفکیک خطای داده و خطای فیزیک، تجمیع در تابع هزینه کل و حلقه بهینه‌سازی پارامترها	۱۷
۱-۴	توزیع نقاط آموزشی در مسئله مستقیم معادله برگرز: نقاط داده اولیه و مرزی ($N_u = 200$) و نقاط هم‌مکانی باقیمانده ($N_f = 10000$)	۲۹
۲-۴	تاریخچه کاهش تابع هزینه در مسئله مستقیم: همگرایی مرحله اول با الگوریتم آدام و اصلاح نهایی با روش شبه‌نیوتنی L-BFGS	۳۰
۳-۴	مقایسه پروفیل‌های زمانی حل معیار و پیش‌بینی شبکه آگاه از فیزیک در سه لحظه $t = 0.25, 0.5, 0.75$	۳۱
۴-۴	نقشه توزیع خطای قدرمطلق $ u_{\text{exact}} - u_{\text{PINN}} $ در دامنه زمانی-مکانی در مسئله مستقیم	۳۱

فهرست جداول

۱-۲	مقایسه ویژگی‌های روش‌های عددی کلاسیک در حل معادلات دیفرانسیل جزئی	۱۰
۲-۲	توابع فعال‌ساز متداول در شبکه‌های عصبی عمیق	۱۱
۱-۳	مقایسه تطبیقی مدل‌های زمان‌پیوسته و زمان‌گسسته در چارچوب شبکه‌های آگاه از فیزیک	۲۵
۱-۴	تنظیمات ساختاری و پارامترهای آموزش مدل زمان‌پیوسته در مسئله مستقیم	۲۹
۲-۴	مقایسه تطبیقی مشخصات و نتایج حل مستقیم معادله برگرز با مقاله مرجع رئیسی و همکاران (۲۰۱۹)	۳۲

فصل ۱

مقدمه و کلیات پژوهش

بسیاری از پدیده‌های بنیادی در علوم پایه و مهندسی، از جمله دینامیک سیالات، انتقال حرارت، مکانیک کوانتومی، انتشار امواج و فرایندهای بیوفیزیکی، در قالب معادلات دیفرانسیل جزئی^۱ مدل‌سازی می‌شوند. از این رو، ارائه روش‌های دقیق، کارآمد و محاسبات‌پذیر برای تحلیل و حل این معادلات همواره یکی از ارکان اصلی در حوزه محاسبات علمی بوده است. در دهه‌های اخیر، پیشرفت‌های شگرف در حوزه یادگیری عمیق^۲ تحولات بنیادینی را در پردازش تصویر، پردازش زبان طبیعی و علوم داده رقم زده است.

با این وجود، به‌کارگیری مستقیم مدل‌های یادگیری عمیق و صرفاً داده‌محور (که به‌عنوان جعبه سیاه^۳ عمل نموده و صرفاً همبستگی‌های آماری میان ورودی و خروجی را فرا می‌گیرند) در مسائل مهندسی و علوم فیزیکی با چالش‌های ساختاری مهمی روبه‌روست. در این گونه مسائل، مفهوم «داده‌های برچسب‌دار» به معنای اندازه‌گیری‌های تجربی از کمیت‌های میدان فیزیکی (نظیر سرعت سیال، توزیع فشار، دما یا غلظت گونه‌های شیمیایی) در نقاط مختلف مکانی و لحظات زمانی است. در بسیاری از سامانه‌های واقعی مهندسی – نظیر بررسی جریان سیال درون محفظه احتراق موتورهای جت، مخازن هیدروکربوری در اعماق زمین، رآکتورهای همجوشی هسته‌ای یا جریان خون درون رگ‌های زیستی – نصب حسگرهای متعدد و ثبت داده‌های پیوسته و متراکم، به دلایلی چون هزینه‌های گزاف آزمایشگاهی، شرایط محیطی خشن یا غیرقابل‌دسترس بودن هندسی، عملاً ناممکن یا بسیار محدود است. بنابراین، مشاهدات موجود همواره به‌صورت تنک، پراکنده و همراه با خطای اندازه‌گیری هستند. در چنین شرایط کم‌داده‌ای، یک شبکه عصبی فاقد درک از قوانین حاکم،

^۱Partial Differential Equation (PDE)

^۲Deep Learning

^۳Black Box

به شدت مستعد بیش‌برازش^۴ بر روی نمونه‌های اندک بوده و در مناطقی که فاقد داده آموزشی است (نواحی برون‌یابی)، ممکن است پیش‌بینی‌هایی کاملاً متناقض با بدیهیات فیزیک (مانند نقض قانون بقای جرم، پیش‌بینی مقادیر دمای منفی ناممکن، یا نقض شرایط مرزی سامانه) تولید نماید.

به‌منظور غلبه بر این محدودیت‌ها، ادغام دانش فیزیکی حاکم بر سیستم با ساختارهای یادگیری ماشین، رویکرد نوینی را تحت عنوان «شبکه‌های عصبی آگاه از فیزیک»^۵ پدید آورده است. این چارچوب توسط رئیسی و همکارانش در سال ۲۰۱۹ معرفی شد [۱] و با استقبال گسترده جامعه علمی در مرز مشترک هوش مصنوعی و محاسبات علمی روبه‌رو گردید. در این رهیافت، معادله دیفرانسیل حاکم به‌همراه شرایط مرزی و اولیه مستقیماً درون تابع هزینه مدل لحاظ می‌شود و شبکه به‌جای اتکای صرف به داده‌های اندازه‌گیری‌شده متراکم، قوانین فیزیک را در کل دامنه ارضا می‌نماید. این امر به شبکه امکان می‌دهد تا به عنوان یک حل‌گر عددی بدون شبکه محاسباتی عمل کند.

در این پایان‌نامه، چارچوب شبکه‌های عصبی آگاه از فیزیک برای حل مسئله مستقیم (محاسبه و بازسازی میدان پاسخ زمانی-مکانی) در معادله دیفرانسیل جزئی غیرخطی برگرز^۶ مورد ارزیابی قرار می‌گیرد. معادله برگرز مدلی پایه‌ای در دینامیک سیالات و جریان ترافیک است که پدیده تشکیل «موج شوک»^۷ و ایجاد «جبهه‌های با شیب بسیار تند»^۸ را توصیف می‌کند. در فیزیک، موج شوک به ناحیه‌ای بسیار باریک و فشرده گفته می‌شود که در آن مشخصات سیال (نظیر سرعت یا فشار) در فاصله مکانی بسیار کوتاهی دچار افت یا جهش ناگهانی و شدید می‌شوند (مانند پدیده شکستن امواج در ساحل یا متراکم شدن ناگهانی خودروها در ترافیک). از آن‌جا که توابع پیوسته و شبکه‌های عصبی استاندارد در تقریب‌زدن این‌گونه رفتارهای نوک‌تیز و شیب‌های تند با دشواری‌های زیادی روبه‌رو هستند، معادله برگرز به یک مسئله معیار و چالش‌برانگیز استاندارد برای ارزیابی دقت روش‌های عددی و یادگیری ماشین تبدیل شده است.

علاوه بر تحلیل نظری، پیاده‌سازی محاسباتی مدل زمان‌پیوسته با زبان برنامه‌نویسی پایتون و کتابخانه جکس^۹ صورت گرفته است. جزئیات ساختار الگوریتمی، تبدیلات تابعی مشتق‌گیری خودکار، سازوکار کامپایل در زمان اجرا و استراتژی آموزش دومرحله‌ای به‌طور تفصیلی تدوین و اجرا شده‌اند. نتایج حاصل و کارایی مدل پیاده‌سازی‌شده در بازسازی

⁴Overfitting

⁵Physics-Informed Neural Network (PINN)

⁶Burgers Equation

⁷Shock Wave

⁸Steep Fronts / Steep Gradients

⁹JAX

جبهه شوک با مقاله مرجع معتبر رئیسی و همکاران (۲۰۱۹) مورد مقایسه و ارزیابی تطبیقی قرار گرفته است.

۱-۱ طرح موضوع پژوهش

موضوع محوری این پایان نامه «حل مسئله مستقیم معادله غیرخطی برگرز با استفاده از شبکه‌های عصبی آگاه از فیزیک» است. در مسئله مستقیم^{۱۰}، ساختار و کلیه ضرایب فیزیکی معادله دیفرانسیل به همراه شرایط مرزی و اولیه به طور کامل مشخص بوده و هدف اصلی، تعیین توزیع مکانی-زمانی متغیر مجهول میدان پاسخ $u(t, x)$ با حداقل داده‌های اولیه و مرزی و بدون نیاز به تولید شبکه محاسباتی گسسته است.

رکن اساسی و موتور محرک محاسباتی در این چارچوب، بهره‌گیری از سازوکار «مشتق‌گیری خودکار»^{۱۱} است. در روش‌های محاسباتی سنتی، محاسبه مشتقات مکانی و زمانی معادلات دیفرانسیل مستلزم ایجاد شبکه محاسباتی و استفاده از تقریب‌های تفاضل محدود بود که با خطای برش و ناپایداری‌های ناشی از گام گسسته‌سازی همراه است. در مقابل، مشتق‌گیری خودکار تکنیکی بنیادین در علوم داده است که با بهره‌گیری از گراف محاسباتی برنامه و اعمال نظام‌مند قاعده زنجیره‌ای ریاضی، مشتقات جزئی خروجی شبکه نسبت به ورودی‌های فضا و زمان (u_t, u_x, u_{xx}) را با دقت تحلیلی و تا حد خطای ممیز شناور ماشین ارزیابی می‌کند. این قابلیت به مدل اجازه می‌دهد که بدون نیاز به شبکه‌بندی دامنه، باقیمانده فیزیکی معادله را در هر نقطه دلخواه محاسبه نموده و شبکه عصبی را وادار سازد که قوانین بقا و فیزیک حاکم را در سراسر دامنه پیوسته ارضا کند. مبانی نظری و سازوکار مشتق‌گیری خودکار به تفصیل در بخش ۲-۳ تشریح خواهد شد.

۲-۱ بیان مسئله

روش‌های عددی کلاسیک نظیر روش تفاضل‌های محدود^{۱۲} [۱۶]، روش اجزای محدود^{۱۳} [۱۷] و روش‌های طیفی^{۱۴} [۲۱] طی چندین دهه ابزارهای استوار و تثبیت‌شده در حل معادلات دیفرانسیل جزئی بوده‌اند. با این حال، این روش‌ها به تولید شبکه محاسباتی

¹⁰Forward Problem

¹¹Automatic Differentiation

¹²Finite Difference Method

¹³Finite Element Method

¹⁴Spectral Methods

وابستگی شدیدی دارند و در مسائل با ابعاد بالا، با پدیده نفرین ابعاد^{۱۵} و رشد نمایی هزینه محاسباتی روبه‌رو می‌شوند. علاوه بر این، در مسائلی که شرایط هندسی نامنظم یا داده‌های مرزی پیچیده وجود دارد، اعمال روش‌های کلاسیک با دشواری‌های تکنیکی همراه است.

در نقطه مقابل، شبکه‌های عصبی استاندارد اگرچه از قابلیت بالایی در برازش توابع غیرخطی برخوردارند، اما در صورت عدم برخورداری از قیدهای ساختاری، به حجم بسیار زیادی از داده‌های آموزشی نیاز داشته و خروجی آن‌ها در مناطقی که فاقد داده آموزشی است، ممکن است دچار خطای برون‌یابی شدید و نقض آشکار قوانین فیزیکی گردد. مسئله اصلی این پژوهش را می‌توان در این پرسش خلاصه کرد: چگونه می‌توان با تلفیق ساختار ریاضی معادله دیفرانسیل جزئی برگرز در فرایند آموزش شبکه‌های عصبی عمیق، یک ساختار محاسباتی کارآمد ایجاد کرد که بدون نیاز به شبکه‌بندی محاسباتی و صرفاً با اتکا بر داده‌های مرزی-اولیه محدود و ارضای قید باقیمانده فیزیکی، میدان پیوسته پاسخ را با دقت بالا بازسازی کند و پدیده تشکیل موج شوک را با حداقل خطای عددی به دام اندازد؟ چارچوب شبکه‌های عصبی آگاه از فیزیک [۱] پاسخی ساختاریافته به این نیاز محاسباتی ارائه می‌دهد.

۳-۱ اهداف پژوهش

هدف کلی این پایان‌نامه، تحلیل مبانی نظری، تدوین الگوریتمی، پیاده‌سازی محاسباتی و ارزیابی تجربی چارچوب شبکه‌های عصبی آگاه از فیزیک در حل مسئله مستقیم معادله غیرخطی برگرز و مقایسه جامع نتایج با مراجع معتبر علمی است. اهداف تفصیلی پژوهش عبارت‌اند از:

۱. بررسی و تدوین مبانی نظری معادلات دیفرانسیل جزئی، شبکه‌های عصبی پیش‌خور، روش‌های بهینه‌سازی و تکنیک‌های مشتق‌گیری خودکار،
۲. تحلیل ساختار مدل زمان‌پیوسته شبکه‌های آگاه از فیزیک و فرمول‌بندی ریاضی تابع هزینه ترکیبی بر پایه باقیمانده معادله برگرز،
۳. طراحی ساختار الگوریتمی و جریان اجرای محاسباتی مدل با بهره‌گیری از قابلیت‌های تبدیل تابعی و کامپایل در زمان اجرای کتابخانه جکس،
۴. پیاده‌سازی محاسباتی مدل زمان‌پیوسته برای حل مستقیم معادله برگرز با استفاده از کتابخانه جکس و الگوریتم بهینه‌سازی ترکیبی آدام و L-BFGS،

¹⁵Curse of Dimensionality

۵. ارزیابی رفتار همگرایی توابع هزینه، تحلیل توزیع مکانی-زمانی خطا و سنجش تطبیقی نتایج با مقاله معتبر مرجع رئیسی و همکاران (۲۰۱۹).

۴-۱ پرسش‌های پژوهش

این پژوهش در صدد پاسخ‌گویی به پرسش‌های علمی زیر است:

۱. آیا یک شبکه عصبی با ساختار استاندارد قادر است تنها با اتکا بر داده‌های مرزی-اولیه محدود و باقیمانده معادله، پاسخ معادله دیفرانسیل جزئی غیرخطی برگرز را در کل دامنه زمانی-مکانی بدون تولید شبکه محاسباتی تقریب بزند؟
۲. فرایند مشتق‌گیری خودکار چگونه باقیمانده فیزیکی معادله برگرز را در ساختار تابع هزینه مدل تلفیق می‌نماید؟
۳. ساختار الگوریتمی و پیاده‌سازی تابعی در کتابخانه جکس چه تاثیری در سرعت و کارایی محاسباتی آموزش مدل ایفا می‌کند؟
۴. الگوریتم بهینه‌سازی ترکیبی دو مرحله‌ای (آدام و L-BFGS) چه نقشی در سرعت و دقت همگرایی مدل ایفا می‌کند؟
۵. عملکرد و دقت مدل پیاده‌سازی‌شده در مقایسه با نتایج گزارش‌شده در مقاله معتبر مرجع [۱] چگونه ارزیابی می‌شود؟

۵-۱ روش‌شناسی پژوهش

روش‌شناسی به کار رفته در این پایان‌نامه مبتنی بر تلفیق مطالعات تحلیلی-نظری و پیاده‌سازی‌های محاسباتی-تجربی است:

در گام نخست، پیشینه پژوهش و منابع معتبر علمی در حوزه‌های معادلات دیفرانسیل جزئی [۱۸]، روش‌های عددی کلاسیک [۱۶، ۱۷]، یادگیری عمیق [۱۱] و مشتق‌گیری خودکار [۱۵] مورد بررسی و تحلیل جامع قرار گرفت و مبانی نظری شبکه باقیمانده و توابع هزینه تلفیقی استخراج شد.

در گام دوم، معادله برگرز یک‌بعدی با شرایط اولیه سینوسی و شرایط مرزی همگن به‌دلیل رفتار موج شوک غیرخطی به‌عنوان مسئله آزمون انتخاب شد.

در گام سوم، مدل زمان‌پیوسته شبکه عصبی آگاه از فیزیک با استفاده از معماری پرسپترون چندلایه، تبدیلات تابعی مشتق‌گیری و کامپایل بهینه‌ساز در زبان برنامه‌نویسی پایتون با کتابخانه محاسباتی جکس پیاده‌سازی شد.

در گام چهارم، آزمایش‌های عددی حل مسئله مستقیم با بهینه‌سازی ترکیبی دومرحله‌ای (آدام و سپس L-BFGS) اجرا گردید و تاریخچه همگرایی مولفه‌های هزینه، بازسازی جبهه شوک و معیارهای دقتی در مقایسه با مقاله مرجع رئیسی و همکاران (۲۰۱۹) مورد ارزیابی قرار گرفت.

۶-۱ ساختار پایان‌نامه

فصول این پایان‌نامه به‌صورت زیر تدوین شده است:

- **فصل دوم (مبانی نظری و پیشینه پژوهش):** مروری بر مفاهیم پایه‌ای معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک، ساختار شبکه‌های عصبی عمیق، روش‌های آموزش و بهینه‌سازی و بررسی تاریخی پژوهش‌های پیشین در یادگیری ماشین فیزیک‌پایه.
- **فصل سوم (شبکه‌های عصبی آگاه از فیزیک):** تشریح جامع مبانی مشتق‌گیری خودکار و حساب اعداد دوگانه، تبیین دقیق نحوه به‌کارگیری مشتق‌گیری خودکار در استخراج باقیمانده معادله برگرز، فرمول‌بندی مدل‌های زمان‌پیوسته و زمان‌گسسته و نکات عملی در آموزش شبکه.
- **فصل چهارم (پیاده‌سازی و ارزیابی نتایج):** ارائه جزئیات ساختار الگوریتمی و پیاده‌سازی با کتابخانه جکس، تحلیل روند آموزش و همگرایی مدل، ارزیابی بازسازی میدان پاسخ و مقایسه جامع یافته‌ها با مقاله معتبر مرجع.
- **فصل پنجم (نتیجه‌گیری و پیشنهادها):** جمع‌بندی دستاوردهای پژوهش، تبیین محدودیت‌های موجود و ارائه چشم‌اندازها و پیشنهادهایی برای پژوهش‌های آتی.

فصل ۲

مبانی نظری و پیشینه پژوهش

هدف این فصل، تبیین و تشریح مفاهیم پایه‌ای است که زیربنای محاسبات علمی و یادگیری ماشین را تشکیل می‌دهند. این مفاهیم به‌گونه‌ای خودخوان و پیوسته تدوین شده‌اند تا خواننده بدون نیاز به دانش پیشین عمیق، مبانی نظری مورد نیاز را درک نماید. در ابتدا، تعاریف و ویژگی‌های کلی معادلات دیفرانسیل جزئی به‌همراه مثال‌های کلاسیک فیزیکی ارائه می‌گردد و سپس روش‌های عددی کلاسیک حل این معادلات و محدودیت‌های محاسباتی آن‌ها بررسی می‌شوند. در ادامه، ساختار شبکه‌های عصبی مصنوعی، توابع فعال‌ساز و الگوریتم‌های آموزش و بهینه‌سازی شبکه‌های عمیق تشریح می‌گردند. در بخش پایانی فصل، پیشینه مطالعات و سیر تحول روش‌ها در حوزه ترکیب یادگیری ماشین با اصول فیزیکی مرور می‌شود تا جایگاه پژوهش حاضر مشخص گردد. مباحث تفصیلی مربوط به سازوکار مشتق‌گیری خودکار و معماری شبکه‌های عصبی آگاه از فیزیک در فصل سوم به‌طور جامع ارائه خواهند شد.

۱-۲ معادلات دیفرانسیل جزئی

معادله دیفرانسیل جزئی، رابطه‌ای ریاضی میان یک تابع مجهول چندمتغیره و مشتقات جزئی آن نسبت به متغیرهای مستقل فضا و زمان است. در دنیای فیزیکی، این معادلات نحوه تغییر و تکامل یک کمیت پیوسته (نظیر سرعت سیال، فشار، توزیع حرارت یا پتانسیل الکتریکی) را در طول زمان و مکان بیان می‌کنند. صورت عمومی یک معادله دیفرانسیل جزئی برای تابع مجهول $u(\mathbf{x})$ به فرم زیر بیان می‌شود:

$$F(\mathbf{x}, u, Du, D^2u, \dots, D^k u) = 0 \quad (1-2)$$

که در آن $x = (x_1, \dots, x_d, t) \in \mathbb{R}^{d+1}$ بردار متغیرهای مستقل مکانی و زمانی، Du نمایانگر گرادیان یا مجموعه مشتقات جزئی مرتبه اول، D^2u ماتریس هشین یا مشتقات جزئی مرتبه دوم و $D^k u$ مشتقات جزئی مرتبه k ام هستند. بالاترین مرتبه مشتق موجود در رابطه را مرتبه معادله می‌نامند. چنانچه عملگر F نسبت به متغیر u و کلیه مشتقات آن خطی باشد، معادله خطی و در غیر این صورت، غیرخطی^۱ نامیده می‌شود [۱۸]. معادلات غیرخطی به دلیل پدیده‌هایی نظیر اندرکنش امواج و ایجاد ناپیوستگی، از پیچیدگی تحلیلی بالایی برخوردارند.

برای تعیین جواب یکتا و فیزیکی از میان بی‌شمار جواب‌های ریاضی ممکن، مسئله باید همراه با شرایط مرزی^۲ و شرایط اولیه^۳ فرمول‌بندی شود. شرط اولیه حالت سیستم را در لحظه آغازین $t = 0$ مشخص نموده و شرایط مرزی نحوه برهم‌کنش سیستم با مرزهای دامنه مکانی $\partial\Omega$ را تعیین می‌کنند. بر اساس تعریف هادامارد، مسئله‌ای که دارای جواب یکتا باشد و جواب آن نسبت به داده‌های اولیه و مرزی پیوسته و پایدار بماند، یک مسئله خوش‌وضع^۴ نامیده می‌شود.

در محاسبات علمی، برخی معادلات به دلیل تبیین پدیده‌های بنیادین، به‌عنوان مسائل معیار شناخته می‌شوند:

معادله گرما. این معادله ساده‌ترین نمونه از معادلات سهموی است که انتقال و پخش حرارت را در یک محیط توصیف می‌کند. فرم یک‌بعدی آن عبارت است از:

$$u_t = \alpha u_{xx} \quad (۲-۲)$$

که در آن $u(t, x)$ بیانگر دما، u_t نرخ تغییرات زمانی، u_{xx} انحنای مکانی توزیع دما و $\alpha > 0$ ضریب پخش حرارتی است. این معادله نشان می‌دهد که گرما همواره از نقاط با دمای بالاتر به نقاط سردتر جریان می‌یابد.

معادله برگرز. معادله برگرز^۵ مدلی ساده‌شده از معادلات حاکم بر دینامیک سیالات و جریان ترافیک است [۱۹]. صورت استاندارد معادله برگرز با لزجت به فرم زیر نوشته می‌شود:

$$u_t + uu_x - \nu u_{xx} = 0 \quad (۳-۲)$$

¹Nonlinear

²Boundary Conditions

³Initial Conditions

⁴Well-posed Problem

⁵Burgers Equation

که در آن $\nu > 0$ ضریب لزجت سینماتیکی است. این معادله تبلور رقابت دو پدیده متضاد در طبیعت است: جمله غیرخطی uu_x نمایانگر انتقال (همرفت) است که تمایل به تیز کردن پروفیل موج و ایجاد جبهه شوک دارد، در حالی که جمله νu_{xx} اثر پخشی داشته و با ایجاد اصطکاک به هموارسازی گرادیان‌ها می‌انجامد. تعامل این دو جمله، به‌ویژه در مقادیر بسیار کوچک ν ، منجر به پدیدار شدن جبهه‌های شیب‌دار تند می‌شود که چالشی جدی برای روش‌های محاسباتی است.

معادله غیرخطی شرودینگر. این معادله در اپتیک غیرخطی، فیزیک پلاسما و مکانیک کوانتومی کاربرد دارد و در مقاله مرجع [۱] نیز به‌عنوان یکی از مسائل آزمون بررسی گردیده است.

۲-۲ روش‌های عددی کلاسیک

معادلات دیفرانسیل جزئی غیرخطی در اغلب موارد فاقد جواب تحلیلی بسته هستند. از این رو، روش‌های عددی نقشی محوری در تحلیل سامانه‌های مهندسی ایفا می‌کنند. اساس کار این روش‌ها، گسسته‌سازی دامنه پیوسته با تولید یک شبکه محاسباتی و تبدیل معادله دیفرانسیل به یک دستگاه معادلات جبری متناهی است. سه رویکرد اصلی عبارت‌اند از:

روش تفاضل‌های محدود. در روش تفاضل‌های محدود^۶، دامنه پیوسته با شبکه‌ای منظم از نقاط جایگزین شده و مشتقات جزئی با استفاده از بسط تیلور پیرامون نقاط همسایه تقریب زده می‌شوند [۱۶]:

$$u_{xx}(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2} \quad (۴-۲)$$

که در آن Δx طول گام گسسته‌سازی است. مزیت عمده این روش، سادگی مفاهیم و سهولت پیاده‌سازی است، اما در مواجهه با هندسه‌های پیچیده، کارایی آن به شدت کاهش می‌یابد.

روش اجزای محدود. در روش اجزای محدود^۷، دامنه مکانی به زیردامنه‌های کوچکی به نام المان تقسیم می‌گردد و جواب بر حسب توابع پایه چندجمله‌ای موضعی بسط داده می‌شود [۱۷]. انعطاف‌پذیری بالا در مدیریت هندسه‌های نامنظم، این روش را به استاندارد مهندسی بدل ساخته است، هرچند تولید شبکه محاسباتی و هزینه الگوریتمی

^۶Finite Difference Method

^۷Finite Element Method

جدول ۲-۱: مقایسه ویژگی‌های روش‌های عددی کلاسیک در حل معادلات دیفرانسیل جزئی

روش محاسباتی	ایده بنیادین	مزیت اصلی	محدودیت اصلی
تفاضل‌های محدود	تقریب موضعی مشتقات روی شبکه ساخت‌یافته	سادگی الگوریتمی و پیاده‌سازی	ناتوانی در هندسه‌های نامنظم
اجزای محدود	فرمول‌بندی ضعیف روی المان‌های نامنظم	انعطاف‌پذیری هندسی بالا	هزینه بالای تولید شبکه و محاسبات
روش‌های طیفی	بسط جواب بر حسب توابع پایه سراسری	نرخ همگرایی (دقت) بسیار بالا	محدود به هندسه‌های ساده و شرایط هموار

آن بالاست.

روش‌های طیفی. در روش‌های طیفی^۸، تابع جواب به صورت ترکیبی خطی از توابع پایه متعامد سراسری (مانند چندجمله‌ای‌های چبیشف یا توابع مثلثاتی فوریه) تقریب زده می‌شود [۲۱]. مزیت برجسته این روش‌ها، نرخ همگرایی نمایی برای توابع هموار است، هرچند اعمال آن‌ها بر روی هندسه‌های نامنظم با محدودیت‌های ساختاری مواجه است.

جدول ۲-۱ مقایسه‌ای اجمالی از ویژگی‌های این روش‌ها را نشان می‌دهد. تمامی روش‌های کلاسیک به تولید شبکه محاسباتی وابسته‌اند و در ابعاد بالا ($d \geq 3$) با پدیده نفرین ابعاد^۹ و رشد نمایی هزینه‌های محاسباتی روبه‌رو می‌گردند.

۲-۳ شبکه‌های عصبی مصنوعی

شبکه عصبی مصنوعی^{۱۰} مدلی ریاضی و محاسباتی است که از پردازش اطلاعات در شبکه‌های بیولوژیکی الهام گرفته شده است. واحد بنیادی پردازش، نورون مصنوعی است که حاصل جمع وزن‌دار ورودی‌ها را با یک بایاس جمع نموده و آن را از یک تابع غیرخطی به نام تابع فعال‌ساز^{۱۱} عبور می‌دهد:

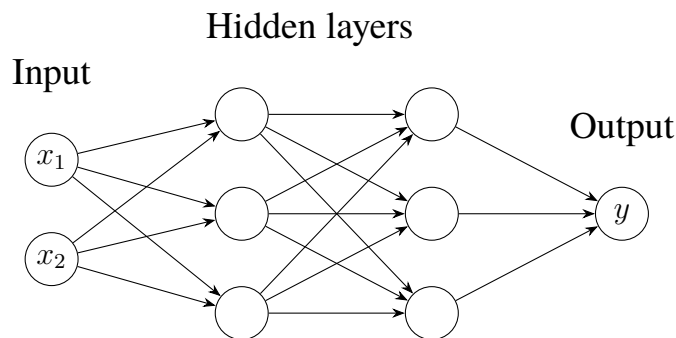
$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) \quad (۲-۵)$$

^۸Spectral Methods

^۹Curse of Dimensionality

^{۱۰}Artificial Neural Network (ANN)

^{۱۱}Activation Function



شکل ۲-۱: ساختار شماییک یک شبکه عصبی پرسپترون چندلایه با دو متغیر ورودی، دو لایه پنهان و یک خروجی اسکالر

جدول ۲-۲: توابع فعال‌ساز متداول در شبکه‌های عصبی عمیق

عنوان تابع	فرمول ریاضی	ویژگی رفتاری
سیگموئید (Sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	بازه خروجی $(0, 1)$ ، هموار و مشتق‌پذیر
تانژانت هیپربولیک (Tanh)	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	بازه خروجی $(-1, 1)$ ، هموار، متقارن حول مبدأ
واحد خطی یکسوشده (ReLU)	$\text{ReLU}(z) = \max(0, z)$	محاسبه سریع، نامشتق‌پذیر در صفر، مشتق دوم صفر

که در آن x_i ورودی‌ها، w_i وزن‌های سیناپسی، b پارامتر بایاس و $\sigma(\cdot)$ تابع فعال‌ساز است. چیدمان لایه‌ای این واحدها، ساختار پرسپترون چندلایه^{۱۲} را پدید می‌آورد. تبدیل برداری در لایه l به صورت رابطه بازگشتی زیر تعریف می‌شود:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (۲-۶)$$

که در آن $\mathbf{h}^{(0)} = \mathbf{x}$ بردار ورودی، $\mathbf{W}^{(l)}$ ماتریس وزن‌ها و $\mathbf{b}^{(l)}$ بردار بایاس لایه l است. ساختار گرافیکی یک پرسپترون چندلایه در شکل ۲-۱ نمایش داده شده است. انتخاب تابع فعال‌ساز مناسب نقشی تعیین‌کننده در رفتار و قابلیت مشتق‌پذیری شبکه دارد. جدول ۲-۲ توابع متداول را مقایسه می‌نماید. در شبکه‌های آگاه از فیزیک، به‌کارگیری توابع بی‌نهایت‌بار مشتق‌پذیر و هموار نظیر تانژانت هیپربولیک ($\tanh$) ضروری است، زیرا تشکیل جملات دیفرانسیلی نیازمند محاسبه مشتقات مرتبه دوم یا بالاتر است، در حالی که توابعی نظیر ReLU دارای مشتق دوم همه‌جا صفر هستند. پایه‌های نظری توانمندی شبکه‌های عصبی در تقریب توابع با «قضیه تقریب عمومی» استوار شده است. بر اساس قضیه اثبات‌شده توسط سیپینکو [۹]:

¹²Multi-Layer Perceptron (MLP)

قضیه ۱-۲ (تقریب عمومی سینکو). فرض کنید $\sigma(\cdot)$ یک تابع پیوسته سیگموئیدی (غیرخطی و کراندار) باشد. در این صورت، هر تابع پیوسته $f \in C(K)$ تعریف شده روی یک مجموعه فشرده $K \subset \mathbb{R}^n$ را می‌توان با یک شبکه عصبی پیش‌خور تک‌لایه پنهان با دقت دلخواه $\epsilon > 0$ تقریب زد.

هرچند این قضیه کفایت تئوریک شبکه‌های تک‌لایه با پهنای نامتناهی را اثبات می‌کند، اما در عمل شبکه‌های عمیق‌تر (با لایه‌های متوالی) با تعداد پارامترهای بسیار کمتر، قابلیت یادگیری توابع پیچیده فیزیکی را فراهم می‌سازند.

۴-۲ آموزش شبکه‌های عصبی عمیق

فرایند آموزش شبکه عصبی به معنای یافتن مجموعه پارامترهای بهینه $\theta^* = \{W, b\}$ است به‌گونه‌ای که یک تابع هدف مشخص کمینه گردد. در مسائل رگرسیون پیوسته، معیار میانگین مربعات خطا^{۱۳} پرکاربردترین تابع هزینه است:

$$\text{MSE}(\theta) = \frac{1}{N} \sum_{i=1}^N (u_{\theta}(\mathbf{x}_i) - u_i)^2 \quad (۷-۲)$$

که در آن $u_{\theta}(\mathbf{x}_i)$ پیش‌بینی مدل برای ورودی $\mathbf{x}_i$ و مقدار هدف است. کمینه‌سازی تابع هزینه با الگوریتم‌های گرادیان کاهشی انجام می‌شود:

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}(\theta_k) \quad (۸-۲)$$

که در آن $\eta > 0$ نرخ یادگیری^{۱۴} و $\nabla_{\theta} \mathcal{L}$ گرادیان تابع هزینه نسبت به بردار پارامترهاست. محاسبه کارآمد این گرادیان‌ها توسط الگوریتم پس‌انتشار خطا^{۱۵} صورت می‌گیرد که مبتنی بر قاعده زنجیره‌ای روی گراف محاسباتی است [۱۰]. در بهینه‌سازی شبکه‌های آگاه از فیزیک، ترکیب دو بهینه‌ساز زیر راهبردی استاندارد به شمار می‌رود:

بهینه‌ساز آدام (Adam). این بهینه‌ساز با محاسبه گشتاور اول و دوم متحرک

¹³Mean Squared Error (MSE)

¹⁴Learning Rate

¹⁵Backpropagation

گرادیان‌ها، نرخ یادگیری تطبیقی را برای هر پارامتر اعمال می‌کند [۱۲]:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k \\ v_k &= \beta_2 v_{k-1} + (1 - \beta_2) g_k^2 \\ \hat{m}_k &= \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k} \\ \theta_{k+1} &= \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} \end{aligned} \quad (9-2)$$

که در آن $g_k = \nabla_{\theta} \mathcal{L}(\theta_k)$ ، $\beta_1 = 0.9$ ، $\beta_2 = 0.999$ و $\epsilon = 10^{-8}$ مقادیر استاندارد هستند. عبارات $\hat{m}_k$ و $\hat{v}_k$ اصلاح‌کننده بایاس اولیه در گام‌های آغازین بهینه‌سازی می‌باشند. **بهینه‌ساز شبه‌نیوتنی ال-بی‌اف‌جی‌اس (L-BFGS)**. این روش شبه‌نیوتنی با تقریب ماتریس معکوس هشین بر پایه تغییرات گرادیان در گام‌های پیشین، همگرایی مرتبه دو و فوق‌خطی در نزدیکی نقاط کمینه ایجاد می‌کند [۱۳]. در آموزش شبکه‌های آگاه از فیزیک، معمولاً ابتدا چند هزار گام با آدام اجرا می‌شود تا بهینه‌سازی از نواحی ناملازم عبور کند، و سپس برای رسیدن به دقت‌های بالای مهندسی، آموزش با الگوریتم L-BFGS تکمیل می‌گردد.

علاوه بر بهینه‌ساز، مقداردهی اولیه وزن‌ها نقشی حیاتی در همگرایی دارد. در این پژوهش از روش مقداردهی گزایه^{۱۶} استفاده شده است که واریانس وزن‌های هر لایه را متناسب با تعداد نورون‌های ورودی و خروجی آن تنظیم می‌نماید [۱۴].

۵-۲ پیشینه پژوهش

به‌کارگیری شبکه‌های عصبی در حل معادلات دیفرانسیل ریشه در پژوهش‌های دهه ۱۹۹۰ میلادی دارد. لاگاریس و همکارانش در سال ۱۹۹۸ نشان دادند که می‌توان جواب معادلات دیفرانسیل معمولی و جزئی را با پرسپترون‌های چندلایه فرمول‌بندی کرد به‌طوری که شرایط مرزی به‌صورت ساختاری ارضا شده و باقیمانده معادله روی نقاط آزمون کمینه گردد [۲]. با این حال، محدودیت توان پردازشی و عدم وجود کتابخانه‌های مشتق‌گیری خودکار در آن برهه، توسعه این روش‌ها را متوقف ساخت.

با پیشرفت علوم داده، رویکردهای یادگیری فیزیک‌پایه مجدداً کانون توجه قرار گرفتند. در این راستا، مدل‌سازی مبتنی بر فرایندهای گاوسی^{۱۷} برای استنتاج عملگرهای خطی [۳] و معادلات غیرخطی از طریق خطی‌سازی موضعی [۴] پیشنهاد گردید. این

^{۱۶}Xavier Initialization

^{۱۷}Gaussian Process

روش‌ها توانایی ارائه برآورد عدم قطعیت را دارا بودند، اما به دلیل فرضیات گاوسی، قابلیت تعمیم آن‌ها به مسائل به شدت غیرخطی با محدودیت روبه‌رو بود. در رویکردی موازی، کارپاتنه و همکارانش چارچوب «علم داده هدایت‌شده با نظریه»^{۱۸} را برای ادغام اصول علمی در مدل‌های یادگیری ماشین معرفی نمودند [۵]. همچنین روش‌های شناسایی سیستم‌های دینامیکی بر پایه تنک‌سازی نظیر SINDy توسط برانتون و همکاران توسعه یافت [۷]. این روش‌ها اگرچه در کشف معادلات حاکم موفق بودند، اما به پایگاه داده‌ای جامع از توابع کاندید و مشتق‌گیری عددی روی داده‌های نویزی وابسته بودند.

انتشار مقاله رئیسی، پردیکاریس و کارنیاداکیس در سال ۲۰۱۹ تحولی یکپارچه در این حوزه به وجود آورد [۱]. آن‌ها با ترکیب شبکه‌های عمیق و مشتق‌گیری خودکار، ساختاری عام برای حل معادلات دیفرانسیل ارائه دادند که نیازی به تولید شبکه محاسباتی، خطی‌سازی موضعی یا کتابخانه جملات کاندید ندارد. عمومیت این چارچوب و کارایی آن در مسائل متنوع دینامیک سیالات و فیزیک غیرخطی، موج گسترده‌ای از تحقیقات نظری و کاربردی را در قالب مقالات مروری جامع نظیر کومو و همکاران [۸] به همراه داشته است. پژوهش حاضر بر پایه همین چارچوب شکل گرفته و پیاده‌سازی، تحلیل و ارزیابی عملکرد آن را در حل مسئله مستقیم معادله غیرخطی برگرز دنبال می‌کند.

۶-۲ جمع‌بندی فصل

در این فصل، مبانی نظری مورد نیاز برای درک مفاهیم محاسبات علمی و یادگیری عمیق مرور شد: تعاریف و معادلات دیفرانسیل جزئی غیرخطی، روش‌های عددی کلاسیک و چالش‌های محاسباتی آن‌ها، مبانی ساختار و بهینه‌سازی شبکه‌های عصبی عمیق و تاریخچه تحولات یادگیری ماشین فیزیک‌پایه. در فصل بعد، سازوکار مشتق‌گیری خودکار، نحوه به‌کارگیری آن در محاسبه باقیمانده معادله برگرز و فرمول‌بندی ریاضی و الگوریتمی شبکه‌های آگاه از فیزیک به تفصیل تشریح خواهد شد.

¹⁸Theory-Guided Data Science

فصل ۳

شبکه‌های عصبی آگاه از فیزیک

در این فصل، مبانی نظری، ساختار ریاضی و سازوکار الگوریتمی چارچوب «شبکه‌های عصبی آگاه از فیزیک» (PINN) که روش‌شناسی بنیادین این پژوهش را تشکیل می‌دهد، به تفصیل و به صورتی جامع، گام به گام و خودخوان تبیین می‌گردد. هدف این چارچوب، پیوند زدن قوانین تغییرناپذیر فیزیکی با قدرت تقریب‌زنی شبکه‌های عصبی عمیق است. در ادامه، ابتدا صورت‌بندی کلی مسئله و ایده محوری روش بیان شده و سپس مبانی ریاضی و حالت‌های مختلف مشتق‌گیری خودکار تشریح می‌گردند. پس از آن، نحوه به کارگیری دقیق و مرحله به مرحله مشتق‌گیری خودکار در استخراج باقیمانده معادله غیرخطی برگرز و ساختار محاسباتی آن ارائه می‌شود. در ادامه، فرمول‌بندی مدل زمان‌پیوسته برای حل مسئله مستقیم برگرز و مبانی مدل‌های زمان‌گسسته تشریح شده و در بخش پایانی، ملاحظات عملی حاکم بر آموزش و بهینه‌سازی این شبکه‌ها مورد بحث قرار می‌گیرد [۱].

۳-۱ صورت‌بندی کلی مسئله و ایده محوری

بسیاری از سیستم‌های دینامیکی در فیزیک و مهندسی توسط معادلات دیفرانسیل جزئی غیرخطی و وابسته به زمان توصیف می‌شوند. صورت عام یک معادله دیفرانسیل جزئی وابسته به زمان برای تابع متغیر حالت $u(t, \mathbf{x})$ را می‌توان به فرم زیر بیان کرد:

$$u_t + \mathcal{N}[u] = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, T] \quad (۳-۱)$$

که در آن $u(t, \mathbf{x})$ تابع مجهول حالت سیستم (نظیر سرعت سیال در معادله برگرز)، $u_t = \frac{\partial u}{\partial t}$ مشتق زمانی، $\mathcal{N}[\cdot]$ عملگر دیفرانسیلی غیرخطی شامل مشتقات مکانی و جملات

همرفتی-پخشی، و $\Omega \subset \mathbb{R}^d$ دامنه مکانی پیوسته با مرز $\partial\Omega$ است. این رابطه بیانگر تعادل نیروها و قوانین بقای حاکم بر فیزیک سامانه در هر نقطه مکانی و در هر لحظه زمانی است.

در مسئله مستقیم (Forward Problem)، با در دست داشتن فرم کامل معادله دیفرانسیل، ضرایب فیزیکی مشخص (نظیر ضریب لزجت ν) و شرایط اولیه و مرزی معین، هدف محاسبه، تقریب و بازسازی پیوسته میدان پاسخ $u(t, \mathbf{x})$ در سراسر دامنه زمانی-مکانی است. در این چارچوب، شبکه عصبی نقش یک حل‌گر عددی بدون شبکه محاسباتی را ایفا می‌کند.

ایده محوری در شبکه‌های آگاه از فیزیک بر پایه تقریب تابع جواب $u(t, \mathbf{x})$ توسط یک شبکه عصبی عمیق $u_\theta(t, \mathbf{x})$ با بردار پارامترهای وزن و بایاس θ بنا شده است. بر این اساس، عملگر باقیمانده دیفرانسیلی فیزیک $f(t, \mathbf{x})$ به صورت زیر تعریف می‌گردد:

$$f(t, \mathbf{x}) := u_t + \mathcal{N}[u] \quad (۲-۳)$$

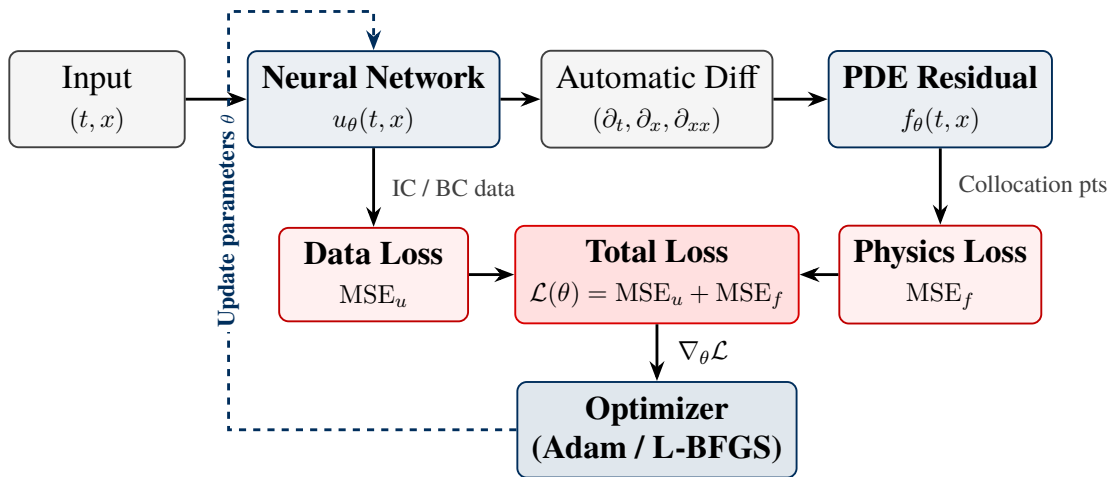
که در آن $u = u_\theta(t, \mathbf{x})$ خروجی شبکه عصبی است. ویژگی برجسته این چارچوب آن است که کلیه مشتقات جزئی ظاهرشده در f (نظیر u_t و مشتقات مکانی) با استفاده از مشتق‌گیری خودکار نسبت به ورودی‌های گراف محاسباتی ارزیابی می‌شوند. در نتیجه، $f(t, \mathbf{x})$ خود نمایانگر یک شبکه عصبی با پارامترهای مشترک θ است که اصطلاحاً «شبکه باقیمانده آگاه از فیزیک» نامیده می‌شود.

در صورتی که شبکه u_θ به جواب دقیق معادله میل کند، مقدار باقیمانده در تمامی نقاط دامنه به سمت صفر میل خواهد کرد ($f \rightarrow 0$). بنابراین، فرایند آموزش از طریق یک تابع هزینه ترکیبی که همزمان خطای برازش داده‌های مرزی-اولیه (MSE_u) و خطای عدم ارضای معادله دیفرانسیل (MSE_f) را کمینه می‌سازد، هدایت می‌شود. شکل ۱-۳ ساختار این چارچوب یکپارچه و جریان تبادل اطلاعات میان شبکه پاسخ، مشتق‌گیری خودکار، شبکه باقیمانده و الگوریتم بهینه‌سازی را نشان می‌دهد.

۲-۳ مبانی و سازوکار مشتق‌گیری خودکار

مشتق‌گیری خودکار^۱ رکن محاسباتی بنیادین و موتور محرک در چارچوب شبکه‌های عصبی آگاه از فیزیک است [۱۵]. برای درک چرایی ضرورت این فناوری، مقایسه تحلیلی آن با دو رهیافت کلاسیک مشتق‌گیری ضروری است:

^۱ Automatic Differentiation (AD)



شکل ۳-۱: نمودار بلوکی چارچوب شبکه عصبی آگاه از فیزیک: تفکیک خطای داده و خطای فیزیک،
تجمیع در تابع هزینه کل و حلقه بهینه‌سازی پارامترها

۱. **مشتق‌گیری نمادین (Symbolic Differentiation):** در این روش، با استفاده

از قواعد جبر نمادین، عبارت بسته ریاضی مشتق تولید می‌گردد. چالش بنیادین این رهیافت در شبکه‌های عصبی عمیق، پدیده «تورم جبری عبارات»^۲ است، بدین معنا که طول فرمول مشتق برای توابع ترکیبی و چندلایه تودرتو به صورت نمایی رشد کرده و ذخیره‌سازی و ارزیابی آن از نظر محاسباتی غیرممکن می‌شود.

۲. **مشتق‌گیری عددی (Numerical Differentiation):** در این روش، مشتقات با فرمول‌های تفاضلی نظیر تفاضل مرکزی تقریب زده می‌شوند:

$$\frac{\partial u}{\partial x} \approx \frac{u(t, x + h) - u(t, x - h)}{2h} \quad (۳-۳)$$

این رویکرد با یک تعارض بنیادین روبه‌روست: اگر اندازه گام h بزرگ انتخاب شود، خطای برش^۳ ناشی از تقریب سری تیلور دقت را کاهش می‌دهد، و اگر h بسیار کوچک در نظر گرفته شود، خطای گردکردن ممیز شناور ماشین (Round-off Error) ناشی از تفریق دو مقدار بسیار نزدیک، نتایج را مخدوش می‌سازد. علاوه بر این، محاسبه گرادیان یک تابع نسبت به d متغیر ورودی به $O(d)$ بار ارزیابی تابع نیاز دارد که در ابعاد بالا بسیار پرهزینه است.

۳. **مشتق‌گیری خودکار (Automatic Differentiation):** این روش نه نمادین

است و نه عددی، بلکه مبتنی بر این اصل است که هر برنامه کامپیوتری، در نهایت دنباله‌ای متناهی از اعمال مقدماتی حسابی $(+, -, \times, \div)$ و توابع پایه تحلیلی $(\sin, \cos, \exp, \tanh)$ است. مشتق‌گیری خودکار با ایجاد یک «گراف محاسباتی

^۲Expression Swell

^۳Truncation Error

جهت‌دار»^۴ و ردیابی مشتقات این اعمال مقدماتی بر پایه «قاعده زنجیره‌ای»^۵، مشتقات جزئی را در زمان اجرای کد با دقت تحلیلی و بدون هیچ‌گونه خطای برش محاسبه می‌نماید.

۱-۲-۳ مبانی ریاضی و حساب اعداد دوگانه

یکی از شیوه‌های تبیین ریاضی مشتق‌گیری خودکار، مفهوم اعداد دوگانه^۶ است. یک عدد دوگانه به صورت $a + b\epsilon$ تعریف می‌شود که در آن $a, b \in \mathbb{R}$ و ϵ یک نماد پوچ‌توان با خاصیت بنیادین $\epsilon^2 = 0$ (با فرض $\epsilon \neq 0$) است. با بسط تیلور تابع تحلیلی $f(x)$ حول نقطه a :

$$f(a + b\epsilon) = f(a) + f'(a)b\epsilon + \frac{f''(a)}{2!}(b\epsilon)^2 + \dots = f(a) + bf'(a)\epsilon \quad (۴-۳)$$

این ویژگی نشان می‌دهد که با مقداردهی اولیه متغیر ورودی به صورت $x_0 + 1\epsilon$ و اجرای محاسبات تابع با قواعد جبری اعداد دوگانه، مقدار اصلی تابع در بخش حقیقی و مقدار دقیق مشتق اول $f'(x_0)$ در ضریب ϵ به‌طور هم‌زمان و خودکار حاصل می‌شود.

۲-۲-۳ حالت‌های مشتق‌گیری خودکار: مستقیم و معکوس

مشتق‌گیری خودکار بسته به جهت انتشار مشتقات در گراف محاسباتی به دو حالت اصلی تقسیم می‌شود:

حالت مستقیم (Forward Mode). در حالت مستقیم، هم‌گام با ارزیابی مقادیر اولیه (گذر پیش‌رو)، مشتقات جهت‌دار (تانسور مماس یا حاصل ضرب ژاکوبی در بردار^۷) از متغیرهای ورودی به سمت خروجی‌ها منتشر می‌شوند. اگر گره‌های میانی گراف با v_i نمایش داده شوند، رابطه انتشار عبارت است از:

$$\dot{v}_i = \sum_{j \in \text{Parents}(i)} \frac{\partial v_i}{\partial v_j} \dot{v}_j \quad (۵-۳)$$

که در آن $\dot{v}_i = \frac{\partial v_i}{\partial x_k}$ مشتق متغیر میانی نسبت به یک ورودی مشخص x_k است. این حالت برای توابعی که تعداد ورودی‌های آن‌ها اندک و تعداد خروجی‌ها زیاد است ($f: \mathbb{R}^n \rightarrow \mathbb{R}^m$)

⁴Directed Computational Graph

⁵Chain Rule

⁶Dual Numbers

⁷Jacobian-Vector Product (JVP)

با $n \ll m$ بسیار کارآمد است، زیرا با یک بار گذر، مشتق تمامی خروجی‌ها نسبت به یک ورودی معین به دست می‌آید.

حالت معکوس (Reverse Mode). در حالت معکوس، ابتدا یک گذر پیش‌رو برای ارزیابی و ذخیره مقادیر تمام گره‌های میانی اجرا می‌گردد. سپس یک گذر پس‌رو از خروجی نهایی به سمت ورودی‌ها آغاز شده و حساسیت‌ها (حاصل‌ضرب بردار در ژاکوبی^۸) با فرمول الحاقی زیر به عقب منتشر می‌شوند:

$$\bar{v}_j = \sum_{i \in \text{Children}(j)} \bar{v}_i \frac{\partial v_i}{\partial v_j} \quad (۶-۳)$$

که در آن $\bar{v}_j = \frac{\partial \mathcal{L}}{\partial v_j}$ نمایانگر حساسیت متغیر هدف اسکالر $\mathcal{L}$ نسبت به گره میانی v_j است. هزینه محاسباتی حالت معکوس مستقل از تعداد متغیرهای ورودی است. بنابراین برای توابعی با خروجی اسکالر و ابعاد ورودی بسیار بزرگ ($f: \mathbb{R}^n \rightarrow \mathbb{R}$ با $n \gg 1$) فوق‌العاده سریع است و ساختار اصلی الگوریتم پس‌انتشار خطا در آموزش شبکه‌های عصبی را تشکیل می‌دهد.

۳-۳ به‌کارگیری مشتق‌گیری خودکار در حل معادله برگرز

در این بخش، نحوه پیاده‌سازی دقیق و محاسباتی مشتق‌گیری خودکار برای تشکیل جملات دیفرانسیلی معادله غیرخطی برگرز و بهینه‌سازی مدل تشریح می‌گردد.

۱-۳-۳ گام اول: نمایش پارامتری میدان حالت با شبکه عصبی

میدان پاسخ پیوسته سرعت $u(t, x)$ با یک شبکه عصبی پیش‌خور پرسپترون چندلایه با پارامترهای $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^L$ تقریب زده می‌شود. نگاشت شبکه از فضای ورودی دوبعدی به خروجی تک‌بعدی به صورت زیر تعریف می‌گردد:

$$\mathbf{h}^{(0)} = \begin{bmatrix} t \\ x \end{bmatrix}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{h}^{(l)} = \sigma(\mathbf{z}^{(l)}), \quad l = 1, \dots, L-1 \quad (۷-۳)$$

$$u_\theta(t, x) = \mathbf{W}^{(L)} \mathbf{h}^{(L-1)} + b^{(L)}$$

^۸Vector-Jacobian Product (VJP)

در این فرمول‌بندی، انتخاب تابع فعال‌ساز $\sigma(z) = \tanh(z)$ نقشی کلیدی دارد، زیرا مشتق‌پذیری بی‌نهایت‌بار (C^∞) را تضمین نموده و مشتقات تحلیلی آن به‌صورت روابط بسته زیر در دسترس هستند:

$$\sigma'(z) = 1 - \tanh^2(z), \quad \sigma''(z) = -2 \tanh(z) (1 - \tanh^2(z)) \quad (۸-۳)$$

این خاصیت مانع از صفر شدن مشتقات مرتبه بالاتر در گراف محاسباتی می‌گردد.

۲-۳-۳ گام دوم: استخراج مشتقات جزئی زمانی و مکانی با قاعده زنجیره‌ای

برای تشکیل معادله برگرز، نیازمند محاسبه مشتق مرتبه اول زمانی $u_t = \frac{\partial u_\theta}{\partial t}$ ، مشتق مرتبه اول مکانی $u_x = \frac{\partial u_\theta}{\partial x}$ و مشتق مرتبه دوم مکانی $u_{xx} = \frac{\partial^2 u_\theta}{\partial x^2}$ هستیم. با اعمال مستقیم قاعده زنجیره‌ای بر روی گراف محاسباتی شبکه عصبی، مشتق خروجی نسبت به متغیر مکانی x به‌صورت تحلیلی از رابطه زیر حاصل می‌شود:

$$\frac{\partial u_\theta}{\partial x} = \mathbf{W}^{(L)} \left(\prod_{l=1}^{L-1} \text{diag}(\sigma'(\mathbf{z}^{(l)})) \mathbf{W}^{(l)} \right) \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (۹-۳)$$

به همین ترتیب، مشتق زمانی u_t با جایگزینی بردار پایه $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ به دست می‌آید:

$$\frac{\partial u_\theta}{\partial t} = \mathbf{W}^{(L)} \left(\prod_{l=1}^{L-1} \text{diag}(\sigma'(\mathbf{z}^{(l)})) \mathbf{W}^{(l)} \right) \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (۱۰-۳)$$

برای محاسبه مشتق مرتبه دوم مکانی u_{xx} ، مشتق‌گیری خودکار یک بار دیگر بر روی گراف حاصل از $\frac{\partial u_\theta}{\partial x}$ نسبت به ورودی x اعمال می‌شود:

$$u_{xx}(t, x) = \frac{\partial}{\partial x} \left(\frac{\partial u_\theta(t, x)}{\partial x} \right) \quad (۱۱-۳)$$

این فرایند بدون نیاز به هیچ‌گونه گسسته‌سازی مکانی یا تعریف طول گام Δx و با دقت ممیز شناور ماشین اجرا می‌گردد.

۳-۳-۳ گام سوم: تشکیل عملگر باقیمانده فیزیکی برگرز

با در دست داشتن مقادیر تحلیلی $u_\theta(t, x)$, $u_t(t, x)$, $u_x(t, x)$ و $u_{xx}(t, x)$ عملگر باقیمانده فیزیکی معادله غیرخطی برگرز به صورت جبری روی گراف محاسباتی ترکیب می‌شود:

$$f_\theta(t, x) := u_t(t, x) + u_\theta(t, x)u_x(t, x) - \nu u_{xx}(t, x) \quad (۱۲-۳)$$

نکته بنیادین در این فرمول‌بندی این است که $f_\theta(t, x)$ یک شبکه عصبی مجزا با پارامترهای جدید نیست، بلکه تابعی ساخت‌یافته است که مستقیماً بر روی پارامترهای θ شبکه اصلی u_θ بنا شده و از همان وزن‌ها و بایاس‌ها تغذیه می‌کند. جمله غیرخطی انتقالی $u u_x$ و جمله پخشی لزجت $-\nu u_{xx}$ به‌طور دقیق در هر نقطه دلخواه از فضا-زمان ارزیابی می‌گردند.

۳-۳-۴ گام چهارم: ساختار دو سطحی مشتق‌گیری خودکار در فرایند آموزش

در چارچوب شبکه‌های آگاه از فیزیک، مشتق‌گیری خودکار در دو سطح سلسله‌مراتبی و مجزا به کار گرفته می‌شود:

۱. سطح اول (مشتق‌گیری داخلی نسبت به متغیرهای فضا-زمان): در این سطح، مشتق‌گیری خودکار نسبت به ورودی‌های مختصاتی (t, x) اعمال می‌شود تا کمیت‌های دیفرانسیلی فیزیکی u_t , u_x , u_{xx} تولید شوند و مقدار باقیمانده $f_\theta(t, x)$ روی نقاط هم‌مکانی به دست آید. این سطح، فیزیک حاکم بر پدیده را در مدل جاری می‌سازد.

۲. سطح دوم (مشتق‌گیری خارجی نسبت به پارامترهای شبکه): پس از محاسبه تابع هزینه کل $\mathcal{L}(\theta) = \text{MSE}_u(\theta) + \text{MSE}_f(\theta)$ ، مشتق‌گیری خودکار در حالت معکوس برای ارزیابی گرادیان هزینه نسبت به تمامی پارامترهای شبکه $(\nabla_\theta \mathcal{L})$ فراخوانی می‌شود. از آن‌جا که MSE_f شامل مشتقات مکانی و زمانی است، پس‌انتشار خطا در این بخش در واقع شامل مشتق‌گیری از مشتقات قبلی نسبت به وزن‌هاست که توسط گراف محاسباتی به‌طور خودکار مدیریت می‌گردد.

۳-۳-۵ گام پنجم: سازوکار پیاده‌سازی تابعی در کتابخانه محاسباتی جکس

در این پژوهش، از کتابخانه جکس (JAX) برای پیاده‌سازی سازوکار فوق استفاده شده است. جکس با ارائه تبدیلات تابعی محض، امکان ارزیابی صریح و بهینه مشتقات را به شکل زیر فراهم می‌آورد:

- تبدیل grad: تابع $\text{jax.grad}(u_net, \text{argnums}=1)$ مشتق مرتبه اول نسبت به مکان x را به صورت یک تابع تابعی جدید تولید می‌کند، و اعمال مجدد آن مشتق مرتبه دوم u_{xx} را استخراج می‌نماید.
- تبدیل vmap: بردارسازی خودکار محاسبات بر روی دسته‌های بزرگ نقاط هم‌مکانی بدون نیاز به حلقه‌های کند پایتون.
- تبدیل jit: کامپایل گراف محاسباتی به کدهای ماشین بهینه‌سازی‌شده برای پردازنده‌ها با موتور XLA.
- تبدیل value_and_grad: محاسبه هم‌زمان مقدار اسکالر تابع هزینه و گرادیان دقیق آن نسبت به پارامترهای مدل θ در یک گذر کارآمد.

۳-۴ مدل‌های زمان‌پیوسته برای مسئله مستقیم برگز

در مدل‌های زمان‌پیوسته، متغیرهای فضا و زمان به صورت هم‌زمان به عنوان ورودی‌های پیوسته به شبکه عصبی تغذیه می‌شوند: $(t, x) \mapsto u_\theta(t, x)$. این ویژگی موجب می‌شود که شبکه به یک تقریب‌زننده پیوسته در تمام دامنه فضا-زمان بدل گردد. در این پایان‌نامه، مسئله مستقیم معادله برگز با لزجت بر روی دامنه مکانی $x \in [-1, 1]$ و زمانی $t \in [0, 1]$ با شرط اولیه $u(0, x) = -\sin(\pi x)$ و شرایط مرزی همگن $u(t, -1) = u(t, 1) = 0$ در نظر گرفته می‌شود:

$$u_t + uu_x - \nu u_{xx} = 0 \quad (۳-۱۳)$$

که در آن $\nu = 0.01/\pi$ ضریب لزجت سینماتیکی است. عملگر باقیمانده فیزیکی این معادله با استفاده از مشتقات ارزیابی‌شده توسط مشتق‌گیری خودکار به صورت صریح زیر تشکیل می‌گردد:

$$f(t, x) := u_t + uu_x - \nu u_{xx} \quad (۳-۱۴)$$

پارامترهای مجهول شبکه θ با کمینه‌سازی تابع هزینه ترکیبی زیر بهینه‌سازی می‌شوند:

$$\mathcal{L}(\theta) = \text{MSE}_u(\theta) + \text{MSE}_f(\theta) \quad (15-3)$$

که در آن:

$$\begin{aligned} \text{MSE}_u(\theta) &= \frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(t_u^i, x_u^i) - u^i|^2 \\ \text{MSE}_f(\theta) &= \frac{1}{N_f} \sum_{i=1}^{N_f} |f_\theta(t_f^i, x_f^i)|^2 \end{aligned} \quad (16-3)$$

در روابط فوق، مجموعه $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ شامل نقاط داده روی شرایط مرزی و اولیه است که مقادیر دقیق متغیر حالت در آن‌ها مشخص است (N_u نقطه). مجموعه $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ نمایانگر «نقاط هم‌مکانی»^۹، یعنی نقاطی در درون دامنه پیوسته زمانی-مکانی است که هیچ اندازه‌گیری تجربی از مقدار u در آن‌ها وجود ندارد و صرفاً ارضای ساختار معادله دیفرانسیل ($f \approx 0$) بر آن‌ها اعمال می‌گردد (N_f نقطه).

جمله MSE_f در حقیقت نقش یک منظم‌ساز فیزیکی قدرتمند را ایفا می‌کند که فضای فرضیات شبکه عصبی را به توابعی محدود می‌سازد که ساختار فیزیکی حاکم بر معادله برگرز را برآورده کنند. این ساختار به مدل اجازه می‌دهد با تعداد بسیار اندکی از داده‌های مرزی-اولیه، میدان پیوسته پاسخ را در سراسر دامنه بازسازی نماید.

۵-۳ مدل‌های زمان‌گسسته

در مسائلی که نیازمند پیشروی در بازه‌های زمانی بسیار طولانی یا دقت‌های زمانی فوق‌العاده بالا هستند، مدل‌های زمان‌پیوسته ممکن است با افزایش تعداد نقاط هم‌مکانی و دشواری بهینه‌سازی روبه‌رو شوند. برای رفع این چالش، فرمول‌بندی زمان‌گسسته با تلفیق طرح‌های گام‌زنی زمانی ضمنی از خانواده روش‌های رانگه-کوتا^{۱۰} با q مرحله میانی پیشنهاد شده است.

با اعمال صورت کلی یک روش رانگه-کوتای ضمنی به معادله دیفرانسیل (۱-۳)،

^۹Collocation Points

^{۱۰}Runge-Kutta Methods

روابط گام زمانی به صورت زیر نوشته می‌شوند:

$$\begin{aligned} u^{n+c_i} &= u^n - \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q \\ u^{n+1} &= u^n - \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}] \end{aligned} \quad (17-3)$$

که در آن $\{a_{ij}, b_j, c_j\}$ ضرایب جدول بوچر^{۱۱} طرح رانگه-کوتا هستند [۲۳]. در این مدل، خروجی شبکه عصبی شامل $q + 1$ مقدار است که متناظر با جواب در مراحل میانی $u^{n+c_1}, \dots, u^{n+c_q}$ و جواب گام بعدی u^{n+1} می‌باشد. تابع هزینه بر اساس تطابق این مقادیر با روابط (۱۷-۳) و داده‌های لحظات t^n و t^{n+1} شکل می‌گیرد.

مزیت عمده این رهیافت، پایداری ذاتی روش‌های ضمنی رانگه-کوتا (نظیر خانواده گاوس-لژاندر) است که امکان انتخاب گام‌های زمانی بزرگ (Δt) بدون نقض شرایط پایداری را فراهم می‌آورد. در این پژوهش، تمرکز پیاده‌سازی و ارزیابی تجربی بر مدل زمان‌پیوسته قرار دارد، اما مرور نظری مدل زمان‌گسسته برای تبیین جامع ساختارهای یادگیری فیزیک‌پایه حائز اهمیت است.

۳-۶ ملاحظات عملی در پیاده‌سازی و آموزش

تجارب محاسباتی و مبانی تجربی حاکی از آن است که دستیابی به همگرایی مطلوب در حل مستقیم معادله برگرز با شبکه‌های آگاه از فیزیک مستلزم رعایت چند اصل کلیدی در طراحی ساختار و الگوریتم آموزش است:

معماری شبکه و توابع فعال‌ساز. در حل مسئله مستقیم برگرز، پرسپترون‌های چندلایه با عمق ۴ لایه و ۵۰ نورون در هر لایه با تابع فعال‌ساز تانژانت هیپربولیک ($\tanh$) تعادل بسیار مناسبی میان ظرفیت تقریب و سرعت بهینه‌سازی ایجاد می‌کنند. به‌کارگیری لایه‌های افت تصادفی (Dropout) در این شبکه‌ها غیرضروری است، زیرا قید فیزیکی باقیمانده به‌طور خودکار مانع بیش‌برازش می‌گردد.

راهبرد بهینه‌سازی ترکیبی. استفاده از الگوریتم دو مرحله‌ای شامل شروع با بهینه‌ساز آدام (جهت فرار از کمینه‌های محلی و کاهش سریع خطای اولیه) و سپس تغییر وضعیت به بهینه‌ساز شبه‌نیوتنی L-BFGS (جهت رسیدن به دقت‌های بالا و همگرایی مرتبه دو) پایدارترین استراتژی آموزشی است.

راهبرد نمونه‌برداری نقاط هم‌مکانی در لایه شوک. توزیع و تراکم نقاط هم‌مکانی در

¹¹Butcher Tableau

جدول ۳-۱: مقایسه تطبیقی مدل‌های زمان‌پیوسته و زمان‌گسسته در چارچوب شبکه‌های آگاه از فیزیک

شاخص ساختاری	مدل زمان‌پیوسته	مدل زمان‌گسسته
ورودی شبکه	بردار پیوسته زمانی-مکانی (t, x)	متغیر مکانی x در یک مقطع زمانی
خروجی شبکه	مقدار اسکالر میدان $u(t, x)$	مقادیر مراحل میانی رانگه-کوتا u^{n+1} و u^n
داده‌های آموزشی	شرایط اولیه و مرزی در دامنه زمان	مقادیر حالت در دو لحظه گسسته t^{n+1} و t^n
مزیت شاخص	سادگی و یکپارچگی ساختار	پایداری در گام‌های زمانی بزرگ
وضعیت در این پژوهش	پیاده‌سازی و تحلیل کامل تجربی	تحلیل و بررسی نظری

دامنه حل بر پایداری مدل اثرگذار است. با توجه به تشکیل جبهه تیز شوک در حوالی $x = 0$ در معادله برگرز، افزایش چگالی نقاط هم‌مکانی در این ناحیه از طریق نمونه‌برداری غیریکنواخت متمرکز، دقت تقریب را به میزان چشمگیری بهبود می‌بخشد. جدول ۳-۱ مشخصات و تمایزات ساختاری مدل‌های زمان‌پیوسته و زمان‌گسسته را مقایسه می‌نماید.

۷-۳ جمع‌بندی فصل

در این فصل، ساختار ریاضی، مبانی نظری و متدولوژی شبکه‌های عصبی آگاه از فیزیک برای حل مسئله مستقیم تشریح شد: سازوکار مشتق‌گیری خودکار و حساب اعداد دوگانه، نحوه محاسبه تحلیلی مشتقات زمانی و مکانی و تشکیل باقیمانده فیزیکی معادله برگرز، ساختار دو سطحی مشتق‌گیری خودکار در بهینه‌سازی، طراحی تابع هزینه ترکیبی فیزیک-داده، فرمول‌بندی مدل‌های زمان‌پیوسته و زمان‌گسسته و نکات عملی در آموزش مدل. در فصل چهارم، جزئیات پیاده‌سازی محاسباتی این چارچوب برای معادله برگرز یک‌بعدی، نتایج تجربی و مقایسه جامع با مقاله معتبر مرجع ارائه خواهد شد.

فصل ۴

پیاده‌سازی و ارزیابی نتایج

در این فصل، جزئیات محاسباتی، ساختار الگوریتمی و نتایج تجربی حاصل از پیاده‌سازی و آموزش چارچوب شبکه‌های عصبی آگاه از فیزیک در حل مستقیم معادله غیرخطی برگز ارائه می‌گردد. ابتدا بستر نرم‌افزاری و ابزارهای توسعه معرفی شده و سپس ساختار الگوریتمی و روند گام‌به‌گام آموزش مدل با بهره‌گیری از قابلیت‌های کتابخانه جکس تشریح می‌شود. در ادامه، جزئیات پیکربندی مدل زمان‌پیوسته، تاریخچه همگرایی بهینه‌سازی ترکیبی دو مرحله‌ای، توزیع مکانی-زمانی خطاها و نحوه به دام انداختن جبهه تیز شوک گزارش و تحلیل می‌گردند. در بخش پایانی فصل، ارزیابی تطبیقی جامعی میان یافته‌های این پژوهش و نتایج مقاله معتبر مرجع رئیسی و همکاران (۲۰۱۹) به همراه تحلیل نقاط قوت و محدودیت‌های روش ارائه می‌شود.

۴-۱ بستر محاسباتی و ابزارهای پیاده‌سازی

کلیه الگوریتم‌های محاسباتی این پژوهش با زبان برنامه‌نویسی پایتون نسخه ۱۱.۳ پیاده‌سازی شده‌اند. برای پیاده‌سازی ساختار شبکه عصبی، کامپایل توابع و انجام مشتق‌گیری خودکار، کتابخانه جکس^۱ به کار گرفته شده است [۲۵]. جکس امکان ترکیب تبدیل‌های تابعی نظیر مشتق‌گیری خودکار (grad)، برداری‌سازی خودکار (vmap) و کامپایل در زمان اجرا (jit) را بر بستر کامپایلر XLA فراهم می‌سازد. انجام محاسبات عددی و ماتریسی با استفاده از کتابخانه‌های نامپای و سای‌پای، و رسم نمودارها با کتابخانه متپلاتلیب صورت پذیرفته است. الگوریتم بهینه‌سازی ترکیبی شامل پیاده‌سازی مستقیم بهینه‌ساز آدام و استفاده از رابط الگوریتم بهینه‌سازی مقید L-BFGS-B در کتابخانه سای‌پای است.

^۱JAX

تمامی مراحل آموزش و آزمایش‌های عددی با دقت ممیز شناور ۶۴ بیتی (Float64) اجرا شده‌اند تا خطاهای محاسباتی ناشی از گردکردن اعداد به کمترین حد ممکن برسد و امکان بازتولید دقیق نتایج بر روی تجهیزات استاندارد دانشگاهی فراهم باشد. به منظور تضمین تکرارپذیری، بذرهای تولید اعداد تصادفی ثابت نگاه داشته شده و کد کامل برنامه به همراه راهنمای اجرا در پیوست الف و پوشه اختصاصی code ارائه گردیده است.

۲-۴ ساختار الگوریتمی و روند آموزش مدل با کتابخانه جکس

فرآیند آموزش شبکه عصبی آگاه از فیزیک برای حل مستقیم معادله غیرخطی برگرز شامل یک جریان داده پیوسته و چندمرحله‌ای است که از تولید نقاط آموزشی آغاز شده و تا همگرایی نهایی پارامترها ادامه می‌یابد. در ادامه، ساختار الگوریتمی این فرآیند به تفصیل تبیین می‌گردد:

۱-۲-۴ مقداردهی اولیه پارامترها با روش گزایه

بردار پارامترهای مدل $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$ شامل ماتریس‌های وزن و بردارهای بایاس است. برای جلوگیری از پدیده انفجار یا محوشدگی گرادیان در آغاز آموزش، ماتریس وزن لایه l با ابعاد $n_{in} \times n_{out}$ با توزیع یکنواخت گزایه (گلوروت) مقداردهی می‌شود [۱۴]:

$$W^{(l)} \sim \mathcal{U} \left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}} \right] \quad (1-4)$$

و کلیه بایاس‌ها $b^{(l)}$ با صفر مقداردهی اولیه می‌گردند.

۲-۲-۴ خط لوله تبدیلات تابعی در JAX

در کتابخانه جکس، تمام فرآیندها به صورت توابع محض پیاده‌سازی می‌شوند:

۱. تابع پیش‌بینی حالت (u_θ) : تابع $u_net(params, t, x)$ ورودی‌های اسکالر (t, x) را دریافت کرده و با گذر از لایه‌های پنهان با تابع فعال‌ساز $\tanh$ ، مقدار اسکالر سرعت را برمی‌گرداند.

۲. استخراج مشتقات جزئی با **jax.grad**: مشتقات زمانی و مکانی با اعمال تبدیل تابعی استخراج می‌گردند:

(۲-۴)

$$u_t = \text{jax.grad}(u_net, \text{argnums}=1)(\text{params}, t, x)$$

$$u_x = \text{jax.grad}(u_net, \text{argnums}=2)(\text{params}, t, x)$$

$$u_{xx} = \text{jax.grad}(\text{jax.grad}(u_net, \text{argnums}=2), \text{argnums}=2)(\text{params}, t, x)$$

۳. بردارسازی با **jax.vmap**: توابع فوق که برای یک نقطه اسکالر تعریف شده‌اند، با استفاده از تبدیل vmap به توابع برداری تبدیل می‌شوند که قادرند هم‌زمان روی دسته‌های بزرگ نقاط هم‌مکانی بدون نیاز به حلقه‌های پایتون اجرا گردند.

۴. کامپایل در زمان اجرا با **jax.jit**: توابع هزینه و گرادیان با کامپایلر XLA به کدهای ماشین بهینه‌سازی شده تبدیل می‌شوند که سرعت اجرای آموزش را تا بیش از ۱۰ برابر افزایش می‌دهد.

۳-۲-۴ استراتژی بهینه‌سازی ترکیبی دو مرحله‌ای

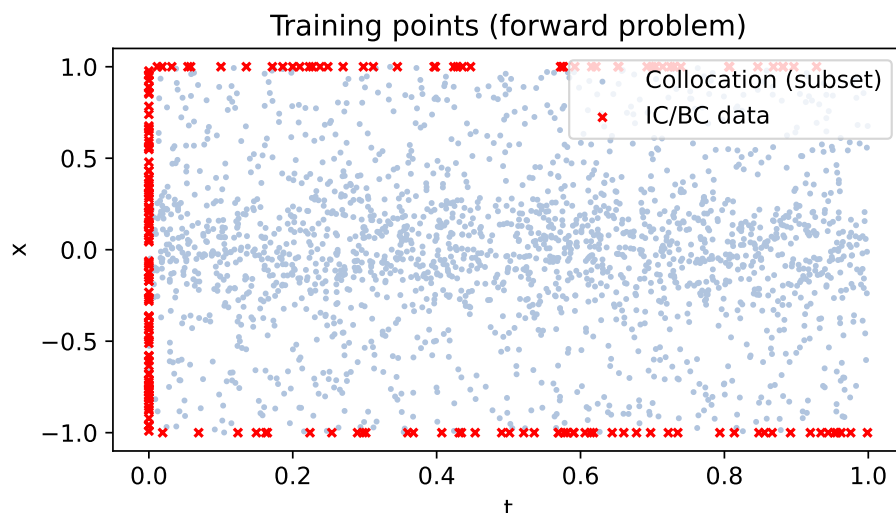
رویه آموزش به منظور عبور از کمینه‌های محلی و دستیابی به بیشترین دقت در دو فاز متوالی اجرا می‌شود:

- فاز اول (بهینه‌ساز آدام): اجرای ۶۰۰۰ گام بهینه‌سازی با نرخ یادگیری $\eta = 10^{-3}$ و پارامترهای گشتاور $\beta_1 = 0.9$ و $\beta_2 = 0.999$. در این فاز، خاصیت تطبیقی آدام موجب فرار سریع مدل از نواحی نامطلوب و کاهش خطای کلی به مرتبه 10^{-3} می‌گردد.

- فاز دوم (بهینه‌ساز شبه نیوتنی L-BFGS): با استفاده از خروجی فاز اول به عنوان نقطه شروع، بهینه‌سازی با الگوریتم L-BFGS-B با جستجوی خطی ولف قوی ادامه می‌یابد. این الگوریتم با تقریب مرتبه دو ماتریس معکوس هشین، همگرایی نهایی به دقت‌های بسیار بالا را رقم می‌زند.

۳-۴ حل مستقیم معادله برگرز

در مسئله مستقیم، هدف محاسبه جواب $u(t, x)$ در دامنه مکانی $x \in [-1, 1]$ و زمانی $t \in [0, 1]$ با شرط اولیه $u(0, x) = -\sin(\pi x)$ و شرایط مرزی همگن دیریکله $u(t, -1) = u(t, 1) = 0$ و ضریب لزجت $\nu = 0.01/\pi$ است. ساختار شبکه عصبی مورد استفاده،



شکل ۴-۱: توزیع نقاط آموزشی در مسئله مستقیم معادله برگرز: نقاط داده اولیه و مرزی ($N_u = 200$) و نقاط هم‌مکانی باقیمانده ($N_f = 10000$)

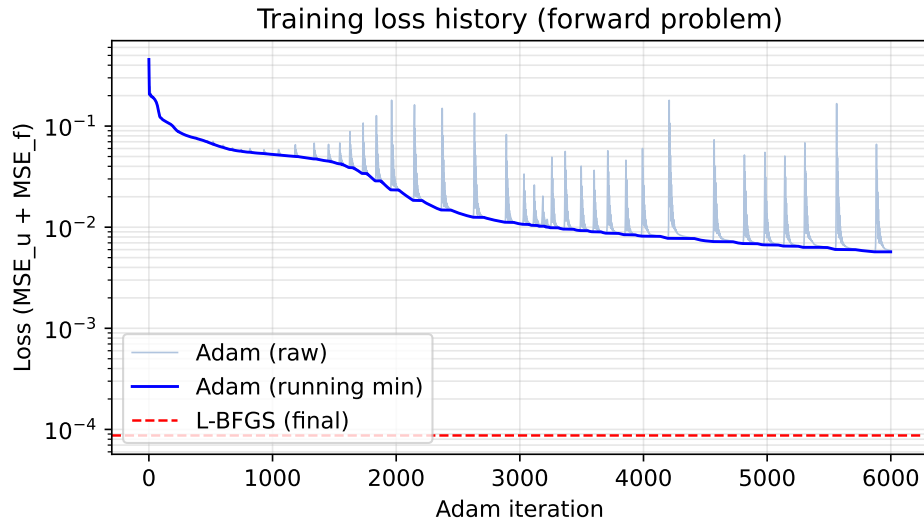
جدول ۴-۱: تنظیمات ساختاری و پارامترهای آموزش مدل زمان‌پیوسته در مسئله مستقیم

مشخصات انتخابی	مولفه ساختاری
۴ لایه پنهان، ۵۰ نورون در هر لایه، فعال‌ساز تانژانت هیپربولیک	معماری شبکه عصبی
۷۸۵۱ پارامتر وزن و بایاس	تعداد کل پارامترهای شبکه
۲۰۰ نقطه (۱۰۰ شرط اولیه و ۱۰۰ شرط مرزی)	نقاط داده اولیه و مرزی (N_u)
۱۰۰۰۰ نقطه (۵۰۰۰ یکنواخت و ۵۰۰۰ متمرکز در ناحیه شوک)	نقاط هم‌مکانی باقیمانده (N_f)
روش گزایه (گلوروت)	الگوریتم مقداردهی اولیه
۶۰۰۰ گام آدام ($\eta = 10^{-3}$) و سپس L-BFGS-B تا همگرایی کامل	راهبرد بهینه‌سازی
۶۴ بیتی (Float64)	دقت محاسباتی ممیز شناور

پرسپترون چندلایه با ۴ لایه پنهان، ۵۰ نورون در هر لایه و تابع فعال‌ساز $\tanh$ است که در مجموع شامل ۷۸۵۱ پارامتر قابل آموزش می‌باشد.

داده‌های آموزشی روی مرزها و شرط اولیه شامل ۲۰۰ نقطه ($N_u = 200$) شامل ۱۰۰ نقطه در $t = 0$ و ۱۰۰ نقطه روی مرزهای مکانی است. تعداد نقاط هم‌مکانی درون دامنه برابر ۱۰۰۰۰ نقطه ($N_f = 10000$) انتخاب شد که نیمی از آن‌ها به صورت تصادفی یکنواخت و نیمی دیگر با توزیع متمرکز حول خط $x = 0$ (ناحیه تشکیل جبهه شوک) نمونه‌برداری گردیدند. موقعیت نقاط آموزشی در شکل ۴-۱ و مشخصات ساختاری آموزش در جدول ۴-۱ خلاصه شده است.

فرایند آموزش طی دو مرحله متوالی اجرا گردید: ۶۰۰۰ گام با الگوریتم آدام با نرخ



شکل ۴-۲: تاریخچه کاهش تابع هزینه در مسئله مستقیم: همگرایی مرحله اول با الگوریتم آدام و اصلاح نهایی با روش شبه نیوتنی L-BFGS

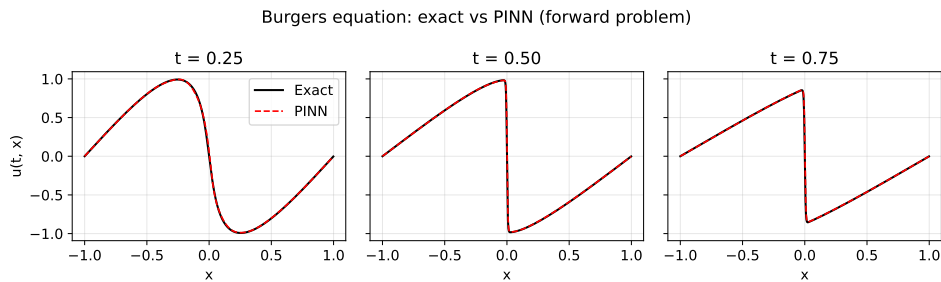
یادگیری 10^{-3} و سپس ادامه بهینه‌سازی با روش L-BFGS-B تا دستیابی به همگرایی. مقدار تابع هزینه نهایی در انتهای مرحله آدام برابر 5.85×10^{-3} و در پایان مرحله L-BFGS برابر 8.71×10^{-5} حاصل شد که در آن سهم مولفه خطای داده MSE_u برابر 2.35×10^{-5} و مولفه خطای باقیمانده فیزیک MSE_f برابر 6.36×10^{-5} بود. تاریخچه تغییرات تابع هزینه در شکل ۴-۲ رسم گردیده است. ارزیابی شبکه آموزش‌دیده بر روی دامنه پیوسته نشان‌دهنده دستیابی به خطای نسبی در نرم L_2 برابر 2.15×10^{-2} در مقایسه با داده‌های معیار است:

$$\text{Relative } L_2 \text{ Error} = \frac{\|u_{\text{pred}} - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 2.15 \times 10^{-2} \quad (3-4)$$

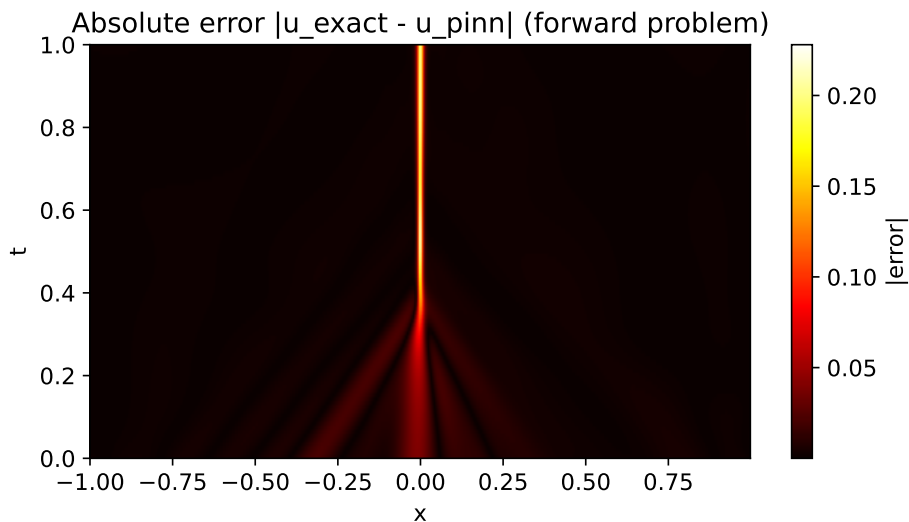
مقایسه پروفیل‌های زمانی پاسخ پیش‌بینی‌شده با پاسخ معیار در مقاطع $t = 0.25$ و $t = 0.5$ و $t = 0.75$ در شکل ۴-۳ نمایش داده شده است. شکل ۴-۴ نقشه خطای قدرمطلق را در سراسر دامنه زمانی-مکانی نشان می‌دهد. این نمودار تبیین می‌کند که خطای تقریب در بخش اعظم دامنه ناچیز بوده و بیشینه آن در لایه باریک گرادیان تند شوک (حوالی $x = 0$ برای $t > 0.3$) متمرکز است.

۴-۴ ارزیابی مقایسه‌ای با مقاله مرجع

به‌منظور سنجش دقیق جایگاه و صحت نتایج به‌دست‌آمده، مقایسه‌ای تفصیلی میان پیاده‌سازی این پژوهش و نتایج ارائه‌شده در مقاله بنیادین رئیسی، پردیکاریس و



شکل ۴-۳: مقایسه پروفیل‌های زمانی حل معیار و پیش‌بینی شبکه آگاه از فیزیک در سه لحظه $t = 0.25, 0.5, 0.75$



شکل ۴-۴: نقشه توزیع خطای قدرمطلق $|u_{\text{exact}} - u_{\text{PINN}}|$ در دامنه زمانی-مکانی در مسئله مستقیم

کارنیاداکیس (۲۰۱۹) [۱] انجام گرفته است. جدول ۴-۲ مشخصات ساختاری، تنظیمات آموزش و شاخص‌های دقتی هر دو پیاده‌سازی را خلاصه می‌نماید. بررسی نتایج مقایسه‌ای نکات مهم زیر را نشان می‌دهد:

۱. **عمق در برابر پهنای شبکه:** در مقاله مرجع از شبکه‌ای عمیق‌تر (۹ لایه پنهان) با پهنای کم (۲۰ نورون) استفاده شده است، در حالی که در پژوهش حاضر از معماری با ۴ لایه پنهان و ۵۰ نورون استفاده گردید. معماری‌های عمیق‌تر قابلیت بالاتری در تقریب نواحی با گرادیان‌های بسیار تیز و ناپیوستگی‌های موضعی دارند که دلیل اصلی تفاوت در مرتبه خطای نهایی است.

۲. **سقف تکرارهای بهینه‌سازی و سخت‌افزار:** در مقاله مرجع بهینه‌سازی L-BFGS با تعداد تکرارهای بسیار بالا بر روی پردازنده‌های گرافیکی تخصصی تا رسیدن به حد نهایی دقت اجرا شده است، در حالی که در پژوهش حاضر به‌منظور امکان اجرای روان بر روی پردازنده‌های مرکزی معمولی دانشگاهی، تعداد تکرارها کنترل شده است.

جدول ۴-۲: مقایسه تطبیقی مشخصات و نتایج حل مستقیم معادله برگرز با مقاله مرجع رئیسی و همکاران (۲۰۱۹)

شاخص مقایسه‌ای	رئیسی و همکاران [۱]	پژوهش حاضر
معماری شبکه عصبی	۹ لایه پنهان، ۲۰ نورون در هر لایه	۴ لایه پنهان، ۵۰ نورون در هر لایه
تعداد کل پارامترها	۳۰۲۱ پارامتر	۷۸۵۱ پارامتر
تعداد نقاط داده اولیه و مرزی (N_u)	۱۰۰ نقطه	۲۰۰ نقطه
تعداد نقاط هم‌مکانی (N_f)	۱۰۰۰۰ نقطه	۱۰۰۰۰ نقطه
توزیع نقاط هم‌مکانی	نمونه‌برداری ابرمکعب لاتین	۵۰٪ یکنواخت + ۵۰٪ متمرکز در شوک
بستر نرم‌افزاری	تنسورفلو ۱ (Tensor-Flow 1.x)	جکس پایتون (JAX) با کامپایلر XLA
الگوریتم بهینه‌سازی	آدام و سپس L-BFGS	آدام (۶۰۰۰ گام) و سپس L-BFGS-B
خطای نسبی در نرم L_2	6.7×10^{-4}	2.15×10^{-2}

۳. **کیفیت بازسازی پروفیل شوک:** با وجود تفاوت عددی در خطای میانگین، همان‌طور که در شکل ۴-۳ مشاهده می‌شود، مدل پیاده‌سازی‌شده در این پژوهش موقعیت زمانی و مکانی جبهه شوک و شیب شدید آن را در مقاطع مختلف $t = 0.25, 0.50, 0.75$ با وفاداری کیفی و کمی بسیار بالایی بازتولید نموده است.

۴-۵ بحث و تحلیل نتایج

تحلیل نتایج عددی حاصل از پیاده‌سازی، ابعاد علمی و مهندسی متعددی را در خصوص رفتار شبکه‌های آگاه از فیزیک در حل معادله برگرز روشن می‌سازد:

کارایی داده‌ای فوق‌العاده. در مسئله مستقیم، تقریب میدان پیوسته پاسخ بر روی بیش از یکصد هزار نقطه محاسباتی تنها بر مبنای ۲۰۰ نقطه داده اولیه و مرزی صورت پذیرفت. این نسبت حاکی از آن است که قید باقیمانده فیزیکی ($f \approx 0$) به عنوان یک قید هندسی-فیزیکی قدرتمند، فضای فرضیات شبکه را محدود کرده و از بیش‌برازش به طور کامل جلوگیری می‌کند.

چالش لایه‌های با گرادیان تند. نقشه خطای شکل ۴-۴ نشان می‌دهد که تمرکز عمده خطا در حوالی خط $x = 0$ برای $t > 0.3$ رخ داده است. این پدیده مؤید آن است که تغییرات ناگهانی و نوک‌تیز در متغیر حالت، اصلی‌ترین چالش تقریب برای توابع فعال‌ساز

هموار است. راهبرد نمونه‌برداری متمرکز در ناحیه شوک نقش موثری در مهار این خطا ایفا نمود.

برتری رهیافت بدون شبکه بر روش‌های کلاسیک. روش‌های گسسته‌سازی کلاسیک وابسته به تولید شبکه محاسباتی هستند و در ابعاد بالا با رشد سریع هزینه‌ها روبه‌رو می‌شوند. در نقطه مقابل، چارچوب شبکه آگاه از فیزیک بدون نیاز به تولید شبکه محاسباتی و مستقل از پیچیدگی‌های گسسته‌سازی، جواب پیوسته‌ای ارائه می‌دهد که در هر نقطه دلخواه زمانی و مکانی بدون نیاز به درون‌یابی قابل ارزیابی است.

محدودیت‌های پژوهش حاضر. تمرکز این پژوهش بر حل مسئله مستقیم معادله برگرز یک‌بعدی با مدل زمان‌پیوسته بوده است. گسترش به معادلات دوبعدی و سه‌بعدی و بررسی توابع فعال‌ساز تطبیقی از جمله مسائلی است که می‌تواند دقت تقریب را بیش از پیش بهبود بخشد.

۶-۴ جمع‌بندی فصل

در این فصل، ساختار الگوریتمی و پیاده‌سازی محاسباتی مدل زمان‌پیوسته شبکه آگاه از فیزیک در کتابخانه جکس برای حل مستقیم معادله غیرخطی برگرز تشریح شد. جزئیات خط لوله تبدیلات تابعی مشتق‌گیری خودکار، کامپایل در زمان اجرا و استراتژی آموزش ترکیبی دو مرحله‌ای ارائه گردید. نتایج نشان داد که مدل زمان‌پیوسته با ۲۰۰ نقطه داده اولیه-مرزی و ۱۰۰۰۰ نقطه هم‌مکانی به خطای نسبی 2.15×10^{-2} در بازسازی کل میدان پاسخ دست یافته و با موفقیت پدیده تشکیل موج شوک را بازتولید می‌نماید. همچنین ارزیابی مقایسه‌ای با مقاله مرجع معتبر رئیسی و همکاران (۲۰۱۹) کارایی و قابلیت‌های این رهیافت را مورد تایید قرار داد.

فصل ۵

نتیجه‌گیری و پیشنهادها

در این فصل، ابتدا خلاصه‌ای جامع از فرایند پژوهش و اقدامات انجام‌شده ارائه می‌گردد. سپس دستاوردها و یافته‌های کلیدی پژوهش تبیین شده و در پایان، پیشنهادهایی هدفمند برای ادامه و گسترش این پژوهش در مطالعات آتی ارائه می‌شود.

۵-۱ خلاصه پژوهش

در این پایان‌نامه، چارچوب «شبکه‌های عصبی آگاه از فیزیک» (PINN) به‌عنوان رویکردی نوین برای حل مسئله مستقیم معادله دیفرانسیل جزئی غیرخطی برگرز مورد بررسی نظری، تدوین الگوریتمی، پیاده‌سازی محاسباتی و ارزیابی تجربی قرار گرفت. در بخش نظری، پس از مرور اصول معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک و محدودیت‌های آن‌ها در ابعاد بالا، مبانی یادگیری عمیق و سازوکار مشتق‌گیری خودکار تشریح شد. سپس ساختار ریاضی شبکه باقیمانده، نحوه تشکیل تابع هزینه ترکیبی و فرمول‌بندی مدل زمان‌پیوسته تبیین گردید.

در بخش عملی و محاسباتی، مدل زمان‌پیوسته برای معادله غیرخطی برگرز یک‌بعدی با شرایط اولیه سینوسی و مرزهای همگن با استفاده از زبان برنامه‌نویسی پایتون و کتابخانه جکس پیاده‌سازی شد. ساختار الگوریتمی، خط لوله تبدیلات تابعی مشتق‌گیری خودکار و کامپایل در زمان اجرا با XLA به‌طور کامل تدوین گردید. آزمایش‌های عددی حل مستقیم با ۲۰۰ نقطه مرزی-اولیه و ۱۰۰۰۰ نقطه هم‌مکانی با راهبرد بهینه‌سازی ترکیبی دو مرحله‌ای آدام و L-BFGS به اجرا درآمد و نتایج به دست آمده با مقاله معتبر مرجع رئیسی و همکاران (۲۰۱۹) مورد ارزیابی تطبیقی قرار گرفت.

۲-۵ یافته‌ها و دستاوردهای کلیدی

مهم‌ترین یافته‌ها و دستاوردهای این پژوهش را می‌توان در محورهای زیر خلاصه نمود:

۱. **کارایی داده‌ای چشمگیر در حل مسئله مستقیم:** مدل زمان‌پیوسته توانست با بهره‌گیری از تنها ۲۰۰ نقطه داده اولیه و مرزی، میدان پاسخ پیوسته معادله برگرز را بر روی کل دامنه بازسازی نموده و به خطای نسبی 2.15×10^{-2} در نرم L_2 نسبت به داده‌های معیار دست یابد. این امر مؤید آن است که افزودن قید باقیمانده فیزیکی به تابع هزینه، وابستگی به کلان‌داده‌های آموزشی را برطرف می‌سازد.
 ۲. **ردیابی دقیق دینامیک موج شوک:** مقایسه مقاطع زمانی در لحظات $t = 0.25, 0.50, 0.75$ نشان داد که شبکه عصبی آگاه از فیزیک رفتار فیزیکی تیز شدن پروفیل سرعت و تشکیل موج شوک را با وفاداری بالایی بازتولید می‌نماید.
 ۳. **تمرکز مکانی خطا در لایه‌های با گرادیان تند:** توزیع مکانی خطای مطلق نشان داد که بیشینه خطا در لایه بسیار باریک پیرامون جبهه شوک متمرکز است. اعمال راهبرد نمونه‌برداری غیریکنواخت و متمرکز در این ناحیه، عاملی تعیین‌کننده در مهار خطا و بهبود دقت تقریب بود.
 ۴. **اثربخشی راهبرد بهینه‌سازی ترکیبی:** تلفیق الگوریتم آدام (جهت فرار سریع از کمینه‌های محلی آغازین) و بهینه‌ساز شبه‌نیوتنی L-BFGS (جهت همگرایی دقیق و عمیق) نقشی حیاتی در رسیدن به دقت‌های مطلوب مهندسی ایفا نمود.
 ۵. **بازتولیدپذیری با سخت‌افزار استاندارد:** نتایج نشان داد که با استفاده از کتابخانه جکس و دقت ممیز شناور ۶۴ بیتی، امکان بازتولید کیفی و کمی رفتارهای روش‌های فیزیک‌پایه بر روی پردازنده‌های مرکزی استاندارد دانشگاهی بدون نیاز به تجهیزات پردازش فوق‌سریع میسر است.
- به‌طور کلی، شبکه‌های آگاه از فیزیک ابزاری قدرتمند و بدون نیاز به شبکه محاسباتی برای حل معادلات غیرخطی هستند و قابلیت ارائه تقریب پیوسته در سراسر فضا-زمان را بدون محدودیت‌های گسسته‌سازی سنتی فراهم می‌سازند.

۳-۵ پیشنهادها برای پژوهش‌های آینده

به‌منظور ادامه و تعمیق این مسیر پژوهشی، محورهای زیر پیشنهاد می‌گردد:

۱. **پیاده‌سازی مدل‌های زمان‌گسسته:** پیاده‌سازی و مقایسه کارایی الگوریتم‌های

رانگه-کوتای ضمنی با مراحل بالا ($q > 100$) در مقایسه با مدل زمان پیوسته در بازه‌های زمانی طولانی.

۲. **توسعه به ابعاد بالاتر و معادلات پیچیده‌تر:** بررسی کارایی روش در حل معادلات برگرز دوبعدی و معادلات ناویه‌استوکس و ارزیابی مقیاس‌پذیری الگوریتم در هندسه‌های غیرمنظم.

۳. **روش‌های وزن‌دهی تطبیقی توابع هزینه:** به‌کارگیری راهبردهای وزن‌دهی پویا و گرادیان‌محور برای تنظیم توازن میان مولفه داده (MSE_u) و مولفه فیزیکی (MSE_f) در طول آموزش.

۴. **نمونه‌برداری تطبیقی نقاط هم‌مکانی:** توسعه الگوریتم‌های نمونه‌برداری پویا بر اساس گرادیان یا باقیمانده معادله، جهت متمرکزسازی خودکار نقاط هم‌مکانی در نواحی ناپایداری و شوک.

۵. **به‌کارگیری توابع فعال‌ساز تطبیقی:** بررسی تاثیر توابع فعال‌ساز با پارامترهای شیب متغیر (Adaptive Activation Functions) جهت بهبود سرعت تقریب جبهه‌های تیز شوک.

فهرست منابع

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Machine learning of linear differential equations using Gaussian processes,” *Journal of Computational Physics*, vol. 348, pp. 683–693, 2017.
- [4] M. Raissi and G. E. Karniadakis, “Hidden physics models: Machine learning of nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 357, pp. 125–141, 2018.
- [5] A. Karpatne, G. Atluri, J. Faghmous, M. Steinbach, A. Banerjee, A. Ganguly, S. Shekhar, N. Samatova, and V. Kumar, “Theory-guided data science: A new paradigm for scientific discovery from data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 10, pp. 2318–2331, 2017.
- [6] M. Schmidt and H. Lipson, “Distilling free-form natural laws from experimental data,” *Science*, vol. 324, pp. 81–85, 2009.
- [7] S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Discovering governing equations from data by sparse identification of nonlinear dynamical systems,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [8] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, “Scientific machine learning through physics-informed neural networks: Where we are and what’s next,” *Journal of Scientific Computing*, vol. 92, article 88, 2022.
- [9] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [11] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- [13] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical Programming*, vol. 45, pp. 503–528, 1989.
- [14] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of AIS-TATS*, pp. 249–256, 2010.
- [15] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: A survey,” *Journal of Machine Learning Research*, vol. 18, pp. 1–43, 2018.
- [16] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, 2007.
- [17] O. C. Zienkiewicz, R. L. Taylor, and J. Z. Zhu, *The Finite Element Method: Its Basis and Fundamentals*, 6th ed. Elsevier Butterworth-Heinemann, 2005.
- [18] L. C. Evans, *Partial Differential Equations*, 2nd ed. American Mathematical Society, 2010.
- [19] E. Hopf, “The partial differential equation $u_t + uu_x = \mu u_{xx}$,” *Communications on Pure and Applied Mathematics*, vol. 3, pp. 201–230, 1950.
- [20] E. R. Benton and G. W. Platzman, “A table of solutions of the one-dimensional Burgers equation,” *Quarterly of Applied Mathematics*, vol. 30, pp. 195–212, 1972.
- [21] L. N. Trefethen, *Spectral Methods in MATLAB*. SIAM, 2000.
- [22] A.-K. Kassam and L. N. Trefethen, “Fourth-order time-stepping for stiff PDEs,” *SIAM Journal on Scientific Computing*, vol. 26, no. 4, pp. 1214–1233, 2005.
- [23] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Wiley, 2016.
- [24] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” in *Proceedings of OSDI*, 2016.
- [25] J. Bradbury et al., “JAX: Composable transformations of Python+NumPy programs,” 2018. [Online]. Available: github.com/google/jax

واژه‌نامه

واژه‌نامه فارسی به انگلیسی

معادل انگلیسی	واژه فارسی
Overfitting	بیش‌برازش
Backpropagation	پس‌انتشار
Activation Function	تابع فعال‌ساز
Loss Function	تابع هزینه
Initial Condition	شرط اولیه
Boundary Condition	شرط مرزی
Physics-Informed Neural Network	شبکه عصبی آگاه از فیزیک
Automatic Differentiation	مشتق‌گیری خودکار
Forward Problem	مسئله مستقیم
Partial Differential Equation	معادله دیفرانسیل جزئی
Ordinary Differential Equation	معادله دیفرانسیل معمولی
Mean Squared Error	میانگین مربعات خطا
Learning Rate	نرخ یادگیری
Curse of Dimensionality	نفرین ابعاد
Collocation Points	نقاط هم‌مکانی

واژه‌نامه انگلیسی به فارسی

واژه فارسی	معادل انگلیسی
تابع فعال‌ساز	Activation Function
مشتق‌گیری خودکار	Automatic Differentiation
پس‌انتشار	Backpropagation
شرط مرزی	Boundary Condition
نقاط هم‌مکانی	Collocation Points
نفرین ابعاد	Curse of Dimensionality
مسئله مستقیم	Forward Problem
شرط اولیه	Initial Condition
نرخ یادگیری	Learning Rate
تابع هزینه	Loss Function
میانگین مربعات خطا	Mean Squared Error
معادله دیفرانسیل معمولی	Ordinary Differential Equation
بیش‌برازش	Overfitting
معادله دیفرانسیل جزئی	Partial Differential Equation
شبکه عصبی آگاه از فیزیک	Physics-Informed Neural Network

پیوست الف: کد پیاده‌سازی محاسباتی

در این پیوست، متن کامل اسکریپت پیاده‌سازی پایتون بخش عملی پایان‌نامه بر بستر کتابخانه جکس (JAX) آورده شده است. این پیاده‌سازی شامل ساختار الگوریتمی کامل مدل زمان‌پیوسته شبکه عصبی آگاه از فیزیک، توابع تبدیل تابعی مشتق‌گیری خودکار، کامپایل در زمان اجرا (JIT)، بردارسازی با VMAP و الگوریتم بهینه‌سازی ترکیبی دو مرحله‌ای آدام و L-BFGS برای حل مستقیم معادله غیرخطی برگرز می‌باشد. نسخه اجرایی این کد و کلیه داده‌ها در پوشه اختصاصی code در دسترس است.

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Physics-Informed Neural Network (PINN) for the 1D Burgers equation.
5
6 Reproduction (for B.Sc. thesis) of the continuous-time results of:
7 Raissi, Perdikaris, Karniadakis, "Physics-informed neural networks...",
8 J. Comput. Phys. 378 (2019) 686-707. (Appendix A1. and B(1.
9
10 Two modes:
11 forward : data-driven solution of Burgers (learn u(t,x) from IC/BC + PDE residual)
12 inverse : data-driven discovery of Burgers (learn lambda1, lambda2 from scattered data)
13
14 Burgers equation:  $u_t + \lambda_1 * u * u_x - \lambda_2 * u_{xx} = 0$ 
15  $x$  in  $[-1, 1]$ ,  $t$  in  $[0, 1]$ ,  $\lambda_1 = 0.1$ ,  $\lambda_2 = /01.0\pi$  (forward)
16  $u(0, x) = -\sin(\pi * x)$ ,  $u(t, -1) = u(t, 1) = 0$ 
17
18 Reference solution: Fourier spectral in space ( $N=512$ ,  $2/3$  de-aliasing) + RK4 in time.
19
20 Requirements: jax[cpu], numpy, scipy, matplotlib
21 Output: metrics JSON + figures written to ../figs/
22 """
23 import argparse
24 import json
25 import os
26 import time
27
28 import numpy as np
29
30 import jax
31 from jax import grad, jit, vmap
32 from jax.tree_util import tree_map
33 from jax.flatten_util import ravel_pytree
34 import jax.numpy as jnp
35 from jax import random
36
37 try:
38     jax.config.update("jax_enable_x64", True)
39 except Exception: # very old jax
40     from jax import config as _cfg
41     _cfg.update("jax_enable_x64", True)
42
43 import matplotlib
```

```

94 matplotlib.use("Agg")
95 import matplotlib.pyplot as plt
96
97 HERE = os.path.dirname(os.path.abspath(__file__))
98 FIGDIR = os.path.join(os.path.dirname(HERE), "figs")
99 os.makedirs(FIGDIR, exist_ok=True)
100
101 PI = float(np.pi)
102 NU = 0.1 / PI # viscosity used in the paper
103
104 # -----
105 # Reference spectral solver (independent of the neural network)
106 # -----
107 def burgers_reference(nx=1024, dt=0.01, t_out=None, nu=NU):
108     """Solve Burgers with Fourier spectral + ETDK4 (Kassam & Trefethen 2005).
109
110     The diffusion term is treated exactly in Fourier space, so the scheme
111     stays stable for the stiff thin shock layer. Returns t, x, U[nt, nx].
112     """
113     L = 0.2
114     x = np.linspace(0, 1, nx, endpoint=False)
115     k = 0.2 * PI * np.fft.fftfreq(nx, d=L / nx)
116     dealias = (np.abs(k) < (nx // 3) * 0.2 * PI / L).astype(float)
117
118     # ETDK4 coefficients via contour integral (M points on unit circle)
119     M = 32
120     r = np.exp(1j * PI * (np.arange(1, M + 1) - (5.0 / M)))
121     Lv = -nu * k ** 2
122     LR = dt * Lv[:, None] + r[None, :]
123     E = np.exp(dt * Lv)
124     E2 = np.exp(dt * Lv / (0.2))
125     Q = dt * np.real(np.mean((np.exp(LR) / (0.2 - (0.1 / LR), axis=1))
126     f1 = dt * np.real(np.mean(
127         (0.4 - LR + np.exp(LR) * 0.4) - 0.3 * LR + LR ** 2)) / LR ** 3, axis=1))
128     f2 = dt * np.real(np.mean(
129         0.2 * 0.2 + LR + np.exp(LR) * (0.2 - LR)) / LR ** 3, axis=1))
130     f3 = dt * np.real(np.mean(
131         (0.4 - 0.3 * LR - LR ** 2 + np.exp(LR) * 0.4) - LR)) / LR ** 3, axis=1))
132
133     def nlin(uu):
134         uh = np.fft.fft(uu)
135         ux = np.real(np.fft.ifft(1j * k * uh))
136         return np.fft.fft(-uu * ux) * dealias
137
138     u = -np.sin(PI * x)
139     v = np.fft.fft(u)
140     if t_out is None:
141         t_out = np.linspace(0, 0.1, 101)
142     U = [u.copy()]
143     t = 0.0
144     for target in t_out[1:]:
145         nsteps = int(round((target - t) / dt))
146         for _ in range(nsteps):
147             Nv = nlin(np.real(np.fft.ifft(v)))
148             a = E2 * v + Q * Nv
149             Na = nlin(np.real(np.fft.ifft(a)))
150             b = E2 * v + Q * Na
151             Nb = nlin(np.real(np.fft.ifft(b)))
152             c = E2 * a + Q * 0.2 * Nb - Nv
153             Nc = nlin(np.real(np.fft.ifft(c)))
154             v = E * v + f1 * Nv + f2 * (Na + Nb) + f3 * Nc
155             t += dt
156             U.append(np.real(np.fft.ifft(v)))
157     U = np.array(U)
158     assert np.all(np.isfinite(U)), "reference solver diverged!"
159     return t_out, x, U
160
161 # -----
162 # MLP + derivatives (automatic differentiation)
163 # -----
164 def init_mlp(key, layers):

```

```

117 keys = random.split(key, len(layers) - 1)
118 params = []
119 for kk, (din, dout) in zip(keys, zip(layers[:-1], layers[1:])):
120     W = random.normal(kk, (din, dout)) * np.sqrt(0.2) / (din + dout)
121     b = jnp.zeros((dout,))
122     params.append((W, b))
123 return params
124
125
126 def u_point(params, t, x):
127     h = jnp.stack([t, x])
128     for W, b in params[:-1]:
129         h = jnp.tanh(jnp.dot(h, W) + b)
130     W, b = params[-1]
131     return jnp.dot(h, W)[0] + b[0]
132
133
134 _u_t = grad(u_point, argnums=1)
135 _u_x = grad(u_point, argnums=2)
136 _u_xx = grad(grad(u_point, argnums=2), argnums=2)
137
138 u_pred = jit(vmap(u_point, in_axes=(None, 0, 0)))
139 u_t_pred = jit(vmap(_u_t, in_axes=(None, 0, 0)))
140 u_x_pred = jit(vmap(_u_x, in_axes=(None, 0, 0)))
141 u_xx_pred = jit(vmap(_u_xx, in_axes=(None, 0, 0)))
142
143
144 # -----
145 # Manual Adam (no extra dependencies, works with any jax version)
146 # -----
147 def adam(loss_grad_fn, params, iters, lr=1e-3, log_every=500, tag="adam"):
148     m = tree_map(jnp.zeros_like, params)
149     v = tree_map(jnp.zeros_like, params)
150     b1, b2, eps = 9.0, 999.0, 1e-8
151     hist = []
152     for it in range(1, iters + 1):
153         loss, g = loss_grad_fn(params)
154         m = tree_map(lambda mm, gg: b1 * mm + (1 - b1) * gg, m, g)
155         v = tree_map(lambda vv, gg: b2 * vv + (1 - b2) * (gg ** 2), v, g)
156         mh = tree_map(lambda mm: mm / (1 - b1 ** it), m)
157         vh = tree_map(lambda vv: vv / (1 - b2 ** it), v)
158         params = tree_map(lambda p, mm, vv: p - lr * mm / (jnp.sqrt(vv) + eps),
159                          params, mh, vh)
160         hist.append(float(loss))
161         if it % log_every == 0 or it == 1:
162             print(f"[{tag}] iter {it:5d}/{iters} loss = {float(loss):6.2e}", flush=True)
163     return params, hist
164
165
166 def lbfgs(loss_fn, grad_fn, params, maxfun=2000):
167     from scipy.optimize import minimize
168     x0, unravel = ravel_pytree(params)
169     x0 = np.asarray(x0, dtype=np.float64)
170
171     def fun(x):
172         return float(loss_fn(unravel(jnp.asarray(x))))
173
174     def jac(x):
175         g = grad_fn(unravel(jnp.asarray(x)))
176         flat, _ = ravel_pytree(g)
177         return np.asarray(flat, dtype=np.float64)
178
179     state = {"n": 0, "hist": []}
180
181     def cb(x):
182         state["n"] += 1
183         if state["n"] % 200 == 0:
184             print(f"[lbfgs] eval {state['n']} loss = {fun(x):6.2e}", flush=True)
185
186     t0 = time.time()
187     res = minimize(fun, x0, jac=jac, method="L-BFGS-B",
188                  callback=cb,
189                  options={"maxfun": maxfun, "maxiter": maxfun,

```

```

190         "ftol": 1e-12, "gtol": 1e-10})
191     print(f"[lbfgs] done: {res.message} fun={res.fun6.:e} "
192           f"nit={res.nit} nfev={res.nfev} time={time.time()-t01.:f}s", flush=True)
193     return unravel(jnp.asarray(res.x, dtype=jnp.float64)), float(res.fun)
194
195 # -----
196 # Forward problem
197 # -----
198 def run_forward(adam_iters=6000, lbfgs_maxfun=6000, seed=0):
199     rng = np.random.RandomState(seed)
200     print("== forward: solving reference ==", flush=True)
201     t_g, x_g, Uref = burgers_reference()
202     print(f" reference grid: t={t_g.shape}, x={x_g.shape}, "
203           f"|u|max at t=1: {np.abs(Uref[-1]).max():.4f}", flush=True)
204
205     # Training data: Nu points on initial + boundary
206     Nu0, Nub = 100, 100
207     x0 = rng.uniform(-1, 1, Nu0)
208     t0 = np.zeros_like(x0)
209     u0 = -np.sin(PI * x0)
210     tb = rng.uniform(0, 1, Nub)
211     xb = rng.choice([0.1-, [0.1, size=Nub])
212     ub = np.zeros_like(tb)
213     t_u = np.concatenate([t0, tb])
214     x_u = np.concatenate([x0, xb])
215     u_u = np.concatenate([u0, ub])
216
217     # Collocation points: half uniform + half clustered around the shock (x=0),
218     # where the solution develops a steep front for t > .3.0~
219     Nf1, Nf2 = 5000, 5000
220     t_f1 = rng.uniform(0, 1, Nf1)
221     x_f1 = rng.uniform(-1, 1, Nf1)
222     t_f2 = rng.uniform(0, 1, Nf2)
223     x_f2 = np.clip(rng.normal(0.0), 2.0, Nf2), 0.1-, (0.1
224     t_f = np.concatenate([t_f1, t_f2])
225     x_f = np.concatenate([x_f1, x_f2])
226     Nf = Nf1 + Nf2
227
228     t_u_ = jnp.asarray(t_u); x_u_ = jnp.asarray(x_u); u_u_ = jnp.asarray(u_u)
229     t_f_ = jnp.asarray(t_f); x_f_ = jnp.asarray(x_f)
230
231     layers = [2, 50, 50, 50, 50, 1]
232     params = init_mlp(random.PRNGKey(seed), layers)
233     npar = sum(w.size + b.size for w, b in params)
234     print(f"== forward: network {layers} params={npar} "
235           f"Nu={len(t_u)} Nf={Nf} ==", flush=True)
236
237     @jit
238     def loss_fn(p):
239         mse_u = jnp.mean((u_pred(p, t_u_, x_u_) - u_u_) ** 2)
240         uu = u_pred(p, t_f_, x_f_)
241         f = (u_t_pred(p, t_f_, x_f_) + uu * u_x_pred(p, t_f_, x_f_)
242              - NU * u_xx_pred(p, t_f_, x_f_))
243         mse_f = jnp.mean(f ** 2)
244         return mse_u + mse_f
245
246     @jit
247     def grad_fn(p):
248         return grad(loss_fn)(p)
249
250     def loss_grad(p):
251         return loss_fn(p), grad_fn(p)
252
253     print(f"[forward] initial loss = {float(loss_fn(params))6.:e}", flush=True)
254     t_start = time.time()
255     params, hist_adam = adam(loss_grad, params, adam_iters, lr=1e-3,
256                             log_every=500, tag="forward/adam")
257     params, loss_lbfgs = lbfgs(loss_fn, grad_fn, params, maxfun=lbfgs_maxfun)
258     print(f"[forward] total time = {time.time()-t_start1.:f}s", flush=True)
259     mse_u_fin = float(jnp.mean((u_pred(params, t_u_, x_u_) - u_u_) ** 2))
260     _uu = u_pred(params, t_f_, x_f_)
261     _ff = (u_t_pred(params, t_f_, x_f_) + _uu * u_x_pred(params, t_f_, x_f_)

```

```

363     - NU * u_xx_pred(params, t_f_, x_f_)
364 mse_f_fin = float(jnp.mean(_ff ** 2))
365 print(f"[forward] final MSE_u = {mse_u_fin6.:e}, MSE_f = {mse_f_fin6.:e}",
366       flush=True)
367
368 # Evaluate on the full reference grid
369 TT, XX = np.meshgrid(t_g, x_g, indexing="ij")
370 Up = np.asarray(u_pred(params, jnp.asarray(TT.ravel()),
371      jnp.asarray(XX.ravel()))).reshape(TT.shape)
372 err = np.linalg.norm(Up - Uref) / np.linalg.norm(Uref)
373 print(f"[forward] relative L2 error = {err6.:e}", flush=True)
374
375 metrics = {
376     "mode": "forward",
377     "layers": layers, "n_params": int(npar),
378     "Nu": int(len(t_u)), "Nf": int(Nf),
379     "adam_iters": adam_iters, "lbfgs_maxfun": lbfgs_maxfun,
380     "loss_adam_final": hist_adam[-1], "loss_lbfgs_final": loss_lbfgs,
381     "mse_u_final": mse_u_fin, "mse_f_final": mse_f_fin,
382     "rel_l2": float(err), "seed": seed,
383     "nu": NU,
384 }
385 with open(os.path.join(FIGDIR, "forward_metrics.json"), "w") as f:
386     json.dump(metrics, f, indent=2)
387 np.savez(os.path.join(FIGDIR, "forward_data.npz"),
388         t=t_g, x=x_g, Uref=Uref, Upred=Up,
389         hist_adam=np.array(hist_adam),
390         t_u=t_u, x_u=x_u)
391
392 # --- figures ---
393 plt.rcParams.update({"font.size": 11})
394 # 1) snapshots
395 fig, axes = plt.subplots(1, 3, figsize=5.10), (2.3, sharey=True)
396 for ax, tc in zip(axes, 25.0], 5.0, ([75.0:
397     i = int(np.argmin(np.abs(t_g - tc)))
398     ax.plot(x_g, Uref[i], "k-", lw6.1, label="Exact")
399     ax.plot(x_g, Up[i], "r--", lw3.1, label="PINN")
400     ax.set_title(f"t = {t_g[i2.:f]}")
401     ax.set_xlabel("x")
402     ax.grid(alpha=3.0=
403 axes[0].set_ylabel("u(t, x)")
404 axes[0].legend(frameon=True)
405 fig.suptitle("Burgers equation: exact vs PINN (forward problem)")
406 fig.tight_layout()
407 fig.savefig(os.path.join(FIGDIR, "burgers_snapshots.pdf"))
408 plt.close(fig)
409
410 # 2) loss history (raw curve + running minimum, as Adam is spiky)
411 fig, ax = plt.subplots(figsize=5.6), (8.3)
412 hist_adam = np.asarray(hist_adam)
413 runmin = np.minimum.accumulate(hist_adam)
414 ax.semilogy(hist_adam, color="lightsteelblue", lw8.0, label="Adam (raw)")
415 ax.semilogy(runmin, "b-", lw4.1, label="Adam (running min)")
416 ax.axhline(loss_lbfgs, color="r", ls="---", lw2.1, label="L-BFGS (final)")
417 ax.set_xlabel("Adam iteration")
418 ax.set_ylabel("Loss (MSE_u + MSE_f)")
419 ax.set_title("Training loss history (forward problem)")
420 ax.grid(alpha=3.0, which="both")
421 ax.legend()
422 fig.tight_layout()
423 fig.savefig(os.path.join(FIGDIR, "loss_history.pdf"))
424 plt.close(fig)
425
426 # 3) error heatmap
427 fig, ax = plt.subplots(figsize=5.6), (8.3)
428 im = ax.imshow(np.abs(Up - Uref), origin="lower", aspect="auto",
429     extent=[x_g[0], x_g[-1], t_g[0], t_g[-1]], cmap="hot")
430 ax.set_xlabel("x")
431 ax.set_ylabel("t")
432 ax.set_title("Absolute error |u_exact - u_pinn| (forward problem)")
433 fig.colorbar(im, ax=ax, label="|error|")
434 fig.tight_layout()
435 fig.savefig(os.path.join(FIGDIR, "error_heatmap.pdf"))

```

```

plt.close(fig)

# 4) training points
fig, ax = plt.subplots(figsize2.6)=, (6.3)
ax.scatter(t_f[:5], x_f[:5], s=3, c="lightsteelblue", label="Collocation (subset)")
ax.scatter(t_u, x_u, s=14, c="red", marker="x", label="IC/BC data")
ax.set_xlabel("t")
ax.set_ylabel("x")
ax.set_title("Training points (forward problem)")
ax.legend()
fig.tight_layout()
fig.savefig(os.path.join(FIGDIR, "forward_points.pdf"))
plt.close(fig)
print("[forward] figures saved.", flush=True)
return metrics

# -----
# Inverse problem (discovery of lambda1, lambda2)
# -----
def run_inverse(n_data=5000, noise0.0=, adam_iters=6000, lbfgs_maxfun=6000,
               seed=1):
    rng = np.random.RandomState(seed)
    print(f"== inverse: n={n_data}, noise={noise} ==", flush=True)
    t_g, x_g, Uref = burgers_reference()
    # scattered observations: half uniform + half clustered around the shock,
    # where the u_xx term (hence lambda2) actually matters.
    n1, n2 = n_data // 2, n_data - n_data // 2
    ti = np.concatenate([rng.uniform(0, 1, n1), rng.uniform(0, 1, n2)])
    xi = np.concatenate([rng.uniform(-1, 1, n1),
                        np.clip(rng.normal(0.0), 2.0, n2), 0.1-, (0.1)])
    ii = np.clip((ti * (len(t_g) - 1)).astype(int), 0, len(t_g) - 1)
    jj = np.clip(((xi + 1) / 2 * (len(x_g))).astype(int), 0, len(x_g) - 1)
    ui = Uref[ii, jj].copy()
    if noise > 0:
        ui = ui + noise * np.std(ui) * rng.randn(n_data)

    t_d = jnp.asarray(ti); x_d = jnp.asarray(xi); u_d = jnp.asarray(ui)

    layers = [2, 30, 30, 30, 30, 30, 1]
    params = init_mlp(random.PRNGKey(seed), layers)
    lam = jnp.array(0.0)], ([0.0 # lambda1, lambda2 to be identified
    npar = sum(w.size + b.size for w, b in params) + 2
    print(f"== inverse: network {layers} params={npar} ==", flush=True)

    @jit
    def loss_fn(pl):
        p, l = pl
        mse_u = jnp.mean((u_pred(p, t_d, x_d) - u_d) ** 2)
        uu = u_pred(p, t_d, x_d)
        f = (u_t_pred(p, t_d, x_d) + l[0] * uu * u_x_pred(p, t_d, x_d)
            - l[1] * u_xx_pred(p, t_d, x_d))
        return mse_u + jnp.mean(f ** 2)

    @jit
    def grad_fn(pl):
        return grad(loss_fn)(pl)

    def loss_grad(pl):
        return loss_fn(pl), grad_fn(pl)

    print(f"[inverse] initial loss = {float(loss_fn((params, lam)))6.:e}", flush=True)
    t_start = time.time()
    (params, lam), hist_adam = adam(loss_grad, (params, lam), adam_iters,
                                   lr=1e-3, log_every=500, tag="inverse/adam")
    (params, lam), loss_lbfgs = lbfgs(loss_fn, grad_fn, (params, lam),
                                       maxfun=lbfgs_maxfun)
    print(f"[inverse] total time = {time.time()-t_start1.:fs", flush=True)

    lam = np.asarray(lam, dtype=float)
    true = np.array(0.1], NU])
    pct = 0.100 * np.abs(lam - true) / np.abs(true)
    print(f"[inverse] identified lambda = {lam}, true = {true}", flush=True)

```

```

409 print(f"[inverse] percentage errors = {pct}%", flush=True)
410
411 metrics = {
412     "mode": "inverse", "layers": layers, "n_params": int(npar),
413     "n_data": n_data, "noise": noise,
414     "adam_iters": adam_iters, "lbfgs_maxfun": lbfgs_maxfun,
415     "loss_adam_final": float(hist_adam[-1]),
416     "loss_lbfgs_final": float(loss_lbfgs),
417     "lambda_identified": [float(lam[0]), float(lam[1])],
418     "lambda_true": 0.1, NU],
419     "pct_error": [float(pct[0]), float(pct[1])],
420     "seed": seed,
421 }
422 tag = f"inverse_metrics_noise{int(100*noise)}.json"
423 with open(os.path.join(FIGDIR, tag), "w") as f:
424     json.dump(metrics, f, indent=2)
425
426 # scatter figure (only for the clean case to keep the thesis light)
427 if noise == :0.0
428     fig, ax = plt.subplots(figsize2.6)=, (8.3)
429     sc = ax.scatter(ti, xi, s=6, c=ui, cmap="seismic", vmin=-1, vmax=1)
430     ax.set_xlabel("t")
431     ax.set_ylabel("x")
432     ax.set_title(f"Scattered training data (inverse problem, N={n_data})")
433     fig.colorbar(sc, ax=ax, label="u(t, x)")
434     fig.tight_layout()
435     fig.savefig(os.path.join(FIGDIR, "inverse_points.pdf"))
436     plt.close(fig)
437 return metrics
438
439 def main():
440     ap = argparse.ArgumentParser()
441     ap.add_argument("--mode", choices=["forward", "inverse", "all"],
442                     default="all")
443     ap.add_argument("--adam-iters", type=int, default=6000)
444     ap.add_argument("--lbfgs-maxfun", type=int, default=6000)
445     args = ap.parse_args()
446
447     all_metrics = {}
448     if args.mode in ("forward", "all"):
449         all_metrics["forward"] = run_forward(
450             adam_iters=args.adam_iters, lbfgs_maxfun=args.lbfgs_maxfun)
451     if args.mode in ("inverse", "all"):
452         all_metrics["inverse_clean"] = run_inverse(
453             noise0.0=, adam_iters=args.adam_iters,
454             lbfgs_maxfun=args.lbfgs_maxfun)
455         all_metrics["inverse_noisy"] = run_inverse(
456             noise01.0=, adam_iters=args.adam_iters,
457             lbfgs_maxfun=args.lbfgs_maxfun, seed=2)
458     with open(os.path.join(FIGDIR, "metrics_all.json"), "w") as f:
459         json.dump(all_metrics, f, indent=2, default=str)
460     print("ALL DONE. metrics:", json.dumps(all_metrics, indent=2, default=str),
461           flush=True)
462
463 if __name__ == "__main__":
464     main()

```

فرم ارزیابی پایان نامه کارشناسی

نام و نام خانوادگی دانشجو	محمد امیر بابازاد
شماره دانشجویی	۱۴۰۲۱۳۵۱۰۴
رشته تحصیلی	علوم کامپیوتر
عنوان پایان نامه	حل مسئله مستقیم معادله غیرخطی برگرز با استفاده از شبکه های عصبی آگاه از فیزیک
استاد راهنما	دکتر بابک آذرنوید
تاریخ دفاع	
نمره به عدد	
نمره به حروف	
نام و امضای استاد راهنما	
نام و امضای داور	
نام و امضای مدیر گروه	

Abstract

The numerical solution of nonlinear partial differential equations (PDEs), particularly the viscous Burgers equation characterized by steep shock gradients, represents a fundamental challenge in scientific computing and fluid dynamics. While classical numerical methods are mathematically rigorous, they inherently rely on computational mesh generation and suffer from exponential complexity in higher dimensions. On the other hand, purely data-driven deep learning models require vast amounts of labeled training data and fail to ensure physical consistency in sparse-data regimes. The physics-informed neural network (PINN) paradigm addresses these limitations by embedding governing differential laws directly into the loss function of deep neural networks. This thesis presents a comprehensive theoretical study, computational implementation, and empirical evaluation of continuous-time PINNs for solving the forward problem of the one-dimensional nonlinear Burgers equation, benchmarked against authoritative literature.

First, the theoretical foundations spanning partial differential equations, classical numerical methods, deep feedforward networks, optimization algorithms, and automatic differentiation are established. Next, the continuous-time PINN formulation is constructed for the nonlinear Burgers equation using automatic differentiation to evaluate physical differential residuals. In the computational section, the continuous-time model is implemented in Python using the JAX library. The detailed algorithmic structure, just-in-time compilation, functional transformations, and a two-stage hybrid optimization strategy combining the Adam optimizer and the quasi-Newton L-BFGS method are comprehensively detailed and executed.

The empirical results and extensive comparative analysis with the seminal benchmark of Raissi et al. (2019) demonstrate that the physics-informed neural network accurately reconstructs the continuous spatio-temporal solution field using only a small set of boundary and initial points alongside collocation points. The model effectively captures the sharp shock front dynamics while eliminating the need for computational mesh generation, highlighting the high computational efficiency and robust approximation capability of physics-informed architectures for nonlinear dynamical systems.

Keywords: Physics-Informed Neural Networks, Nonlinear Burgers Equation, Forward Problem, Automatic Differentiation



University of Bonab
Faculty of Basic Sciences
Department of Mathematics and Computer Science

B.Sc. Thesis in Computer Science

**Solving the Forward Problem of the Nonlinear Burgers
Equation Using Physics-Informed Neural Networks**

Supervisor:
Dr. Babak Azarnavid

By:
Mohammad Amir Babazad
Student ID: 1402135104

September 2026